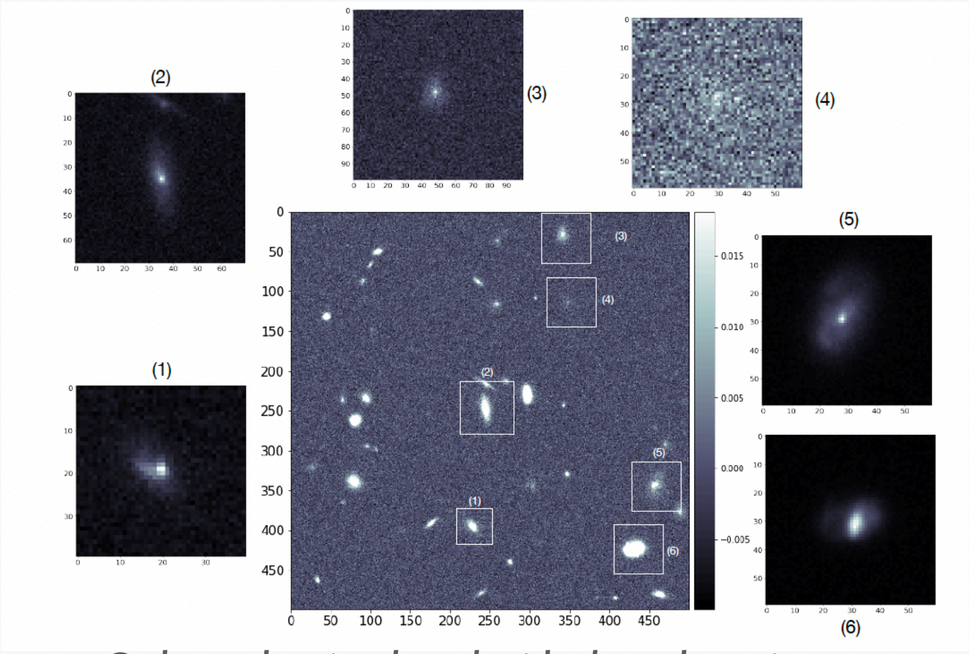
Introduction to deep learning for biomedical researchers: from neurons to LLMs.

Hubert Bretonnière, CCB–VHIO

hubertbretonniere@vhio.net

A few words about me

PhD in astrophysics in 2019

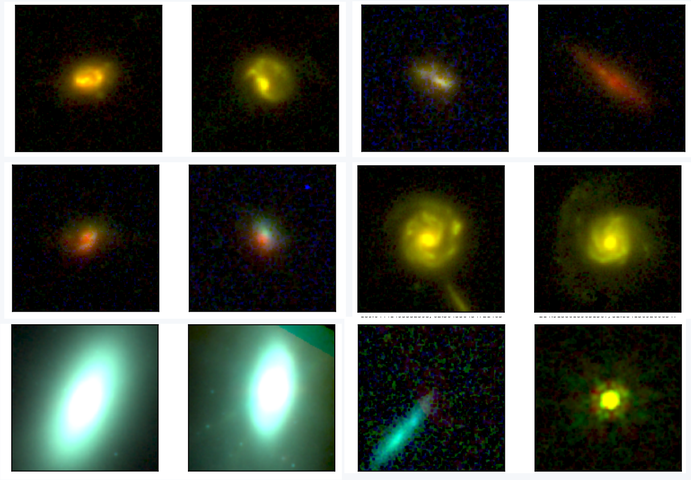


Galaxy sky simulated with deep learning

France

Tenerife

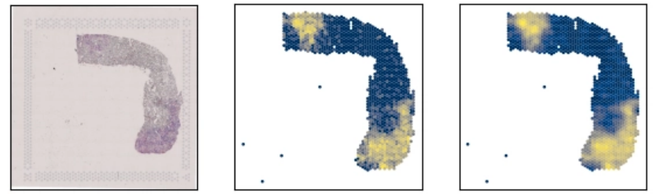
Postdoc at UCSC in 2022



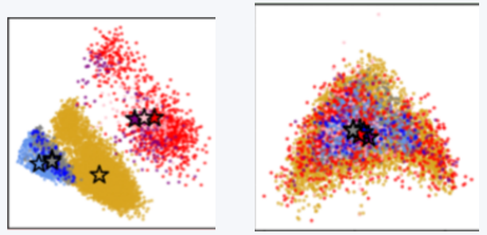
Outlier detections with deep learning

Santa Cruz, California

Postdoc at VHIO CCB since 2024



Spatial Transcriptomic prediction



Batch removal models

Barcelona, Spain

A few words about you

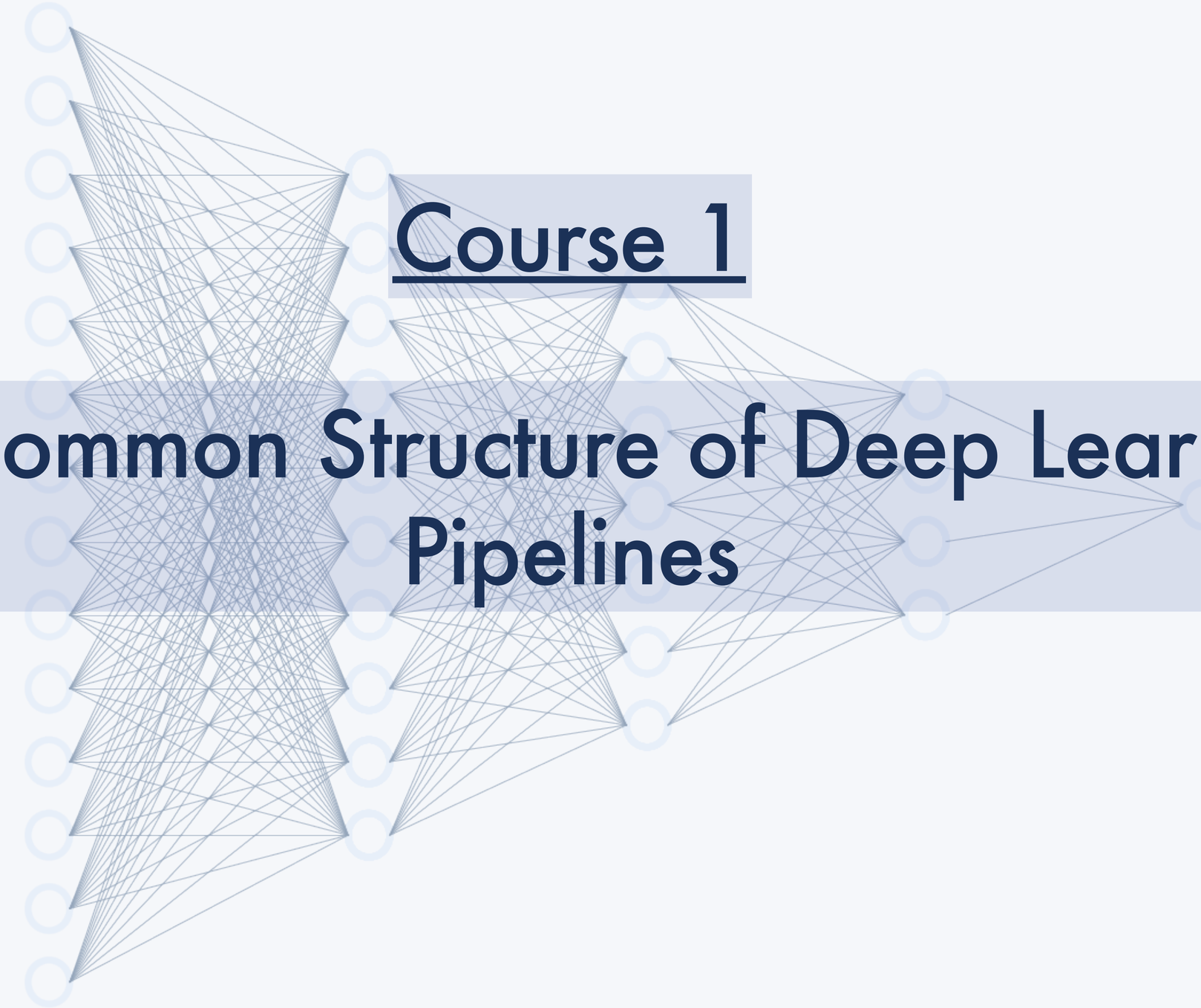
Mentimeter link

Lecture Plan:

Course 1: The Common Structure of Deep Learning Pipelines

Course 2: Convolution and Transformers: Going Deeper into Deep Learning

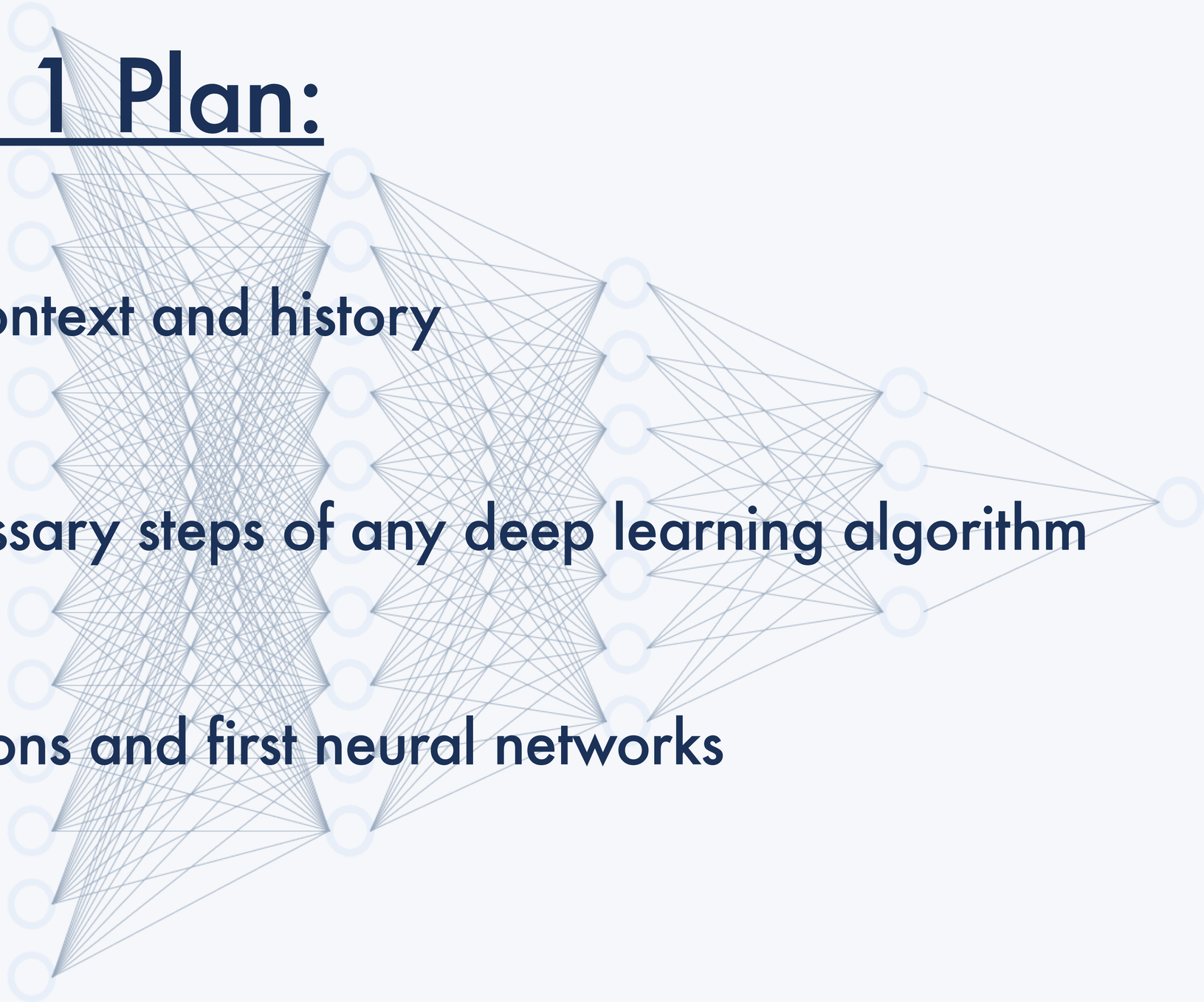
Course 3: Unsupervised learning: From Latent Spaces to Foundation Models



Course 1

The Common Structure of Deep Learning Pipelines

Course 1 Plan:

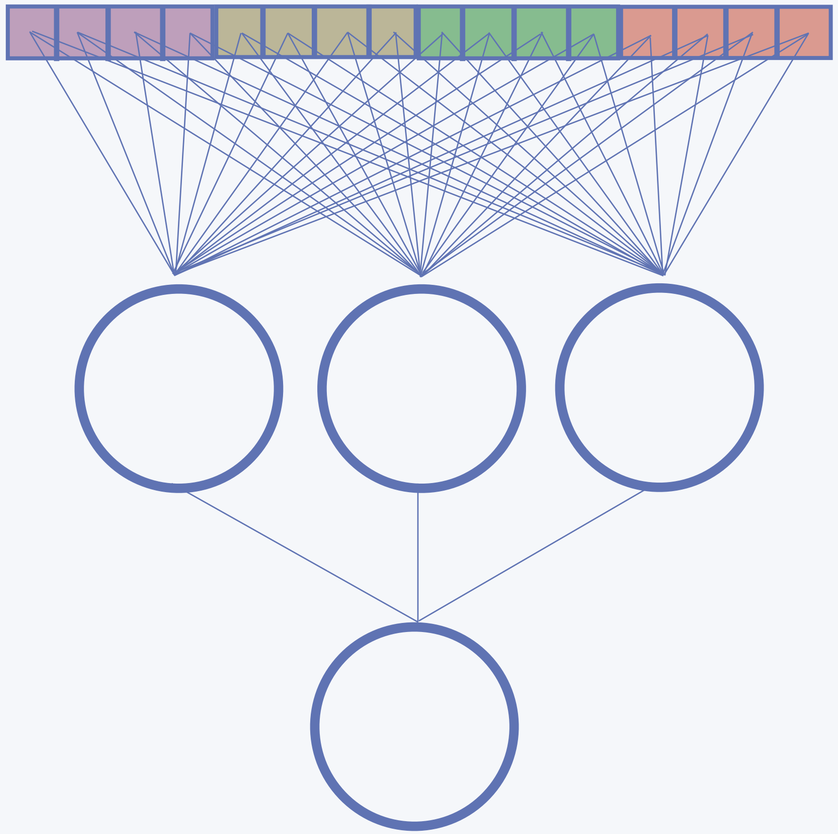


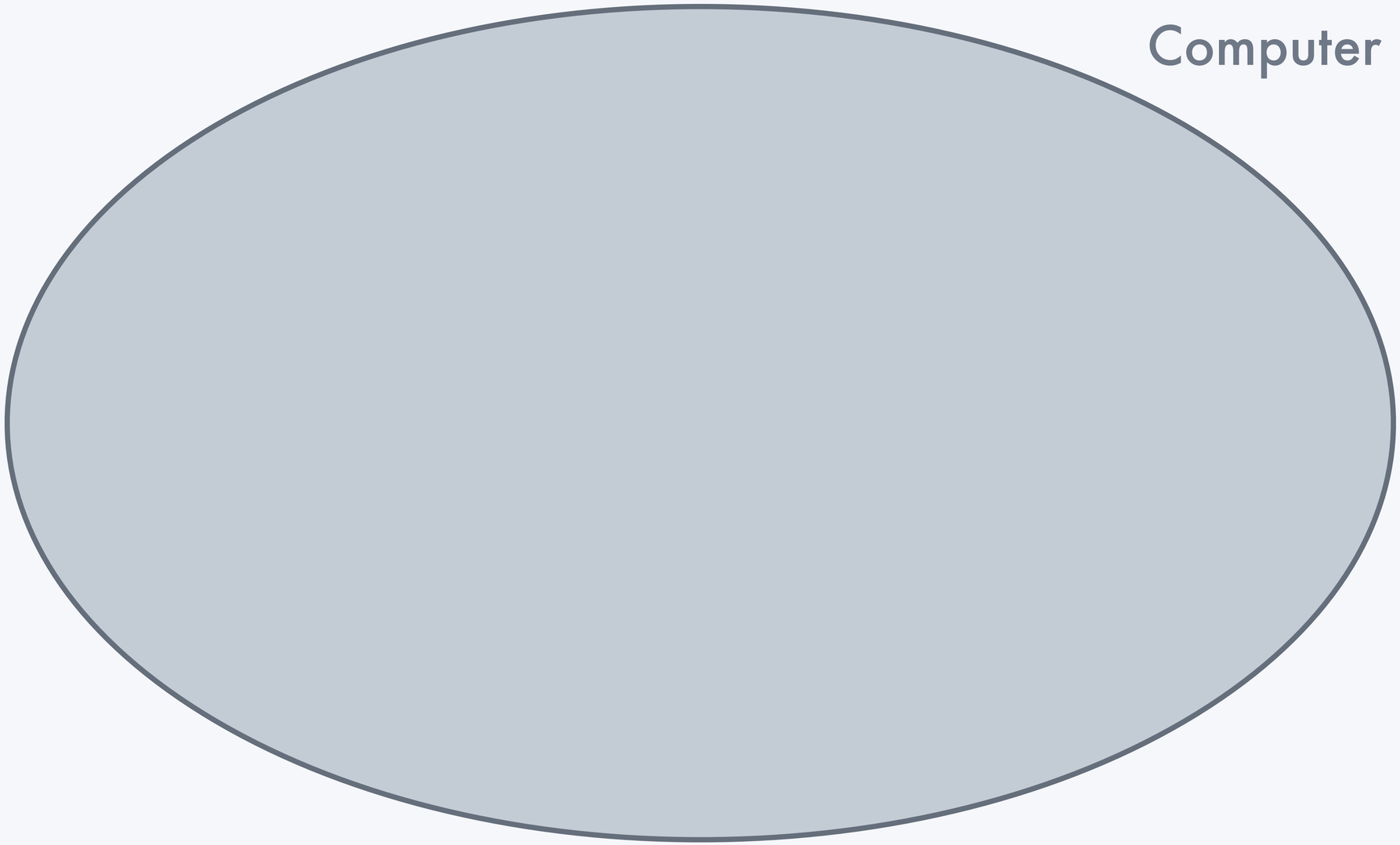
I) A bit of context and history

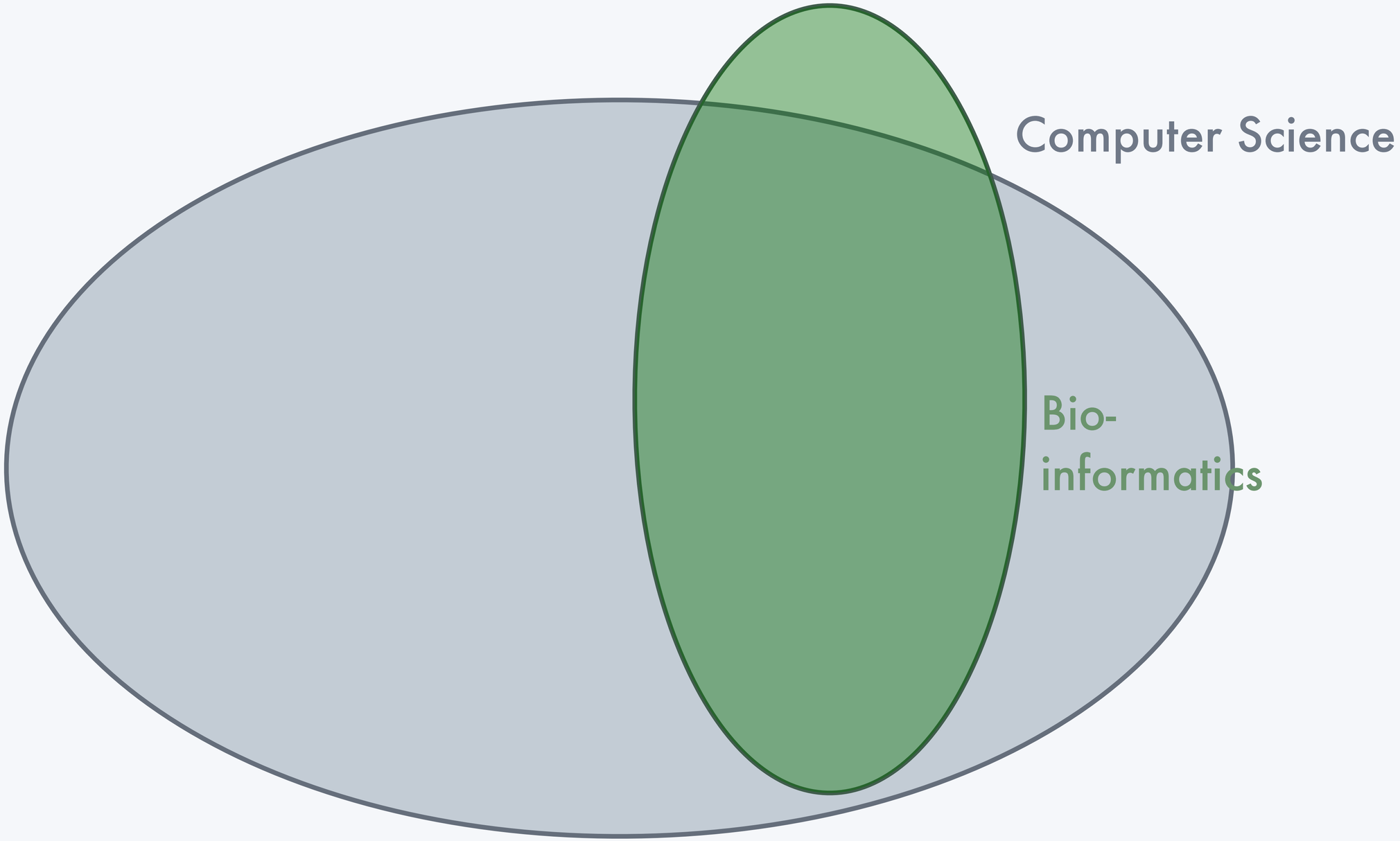
II) The necessary steps of any deep learning algorithm

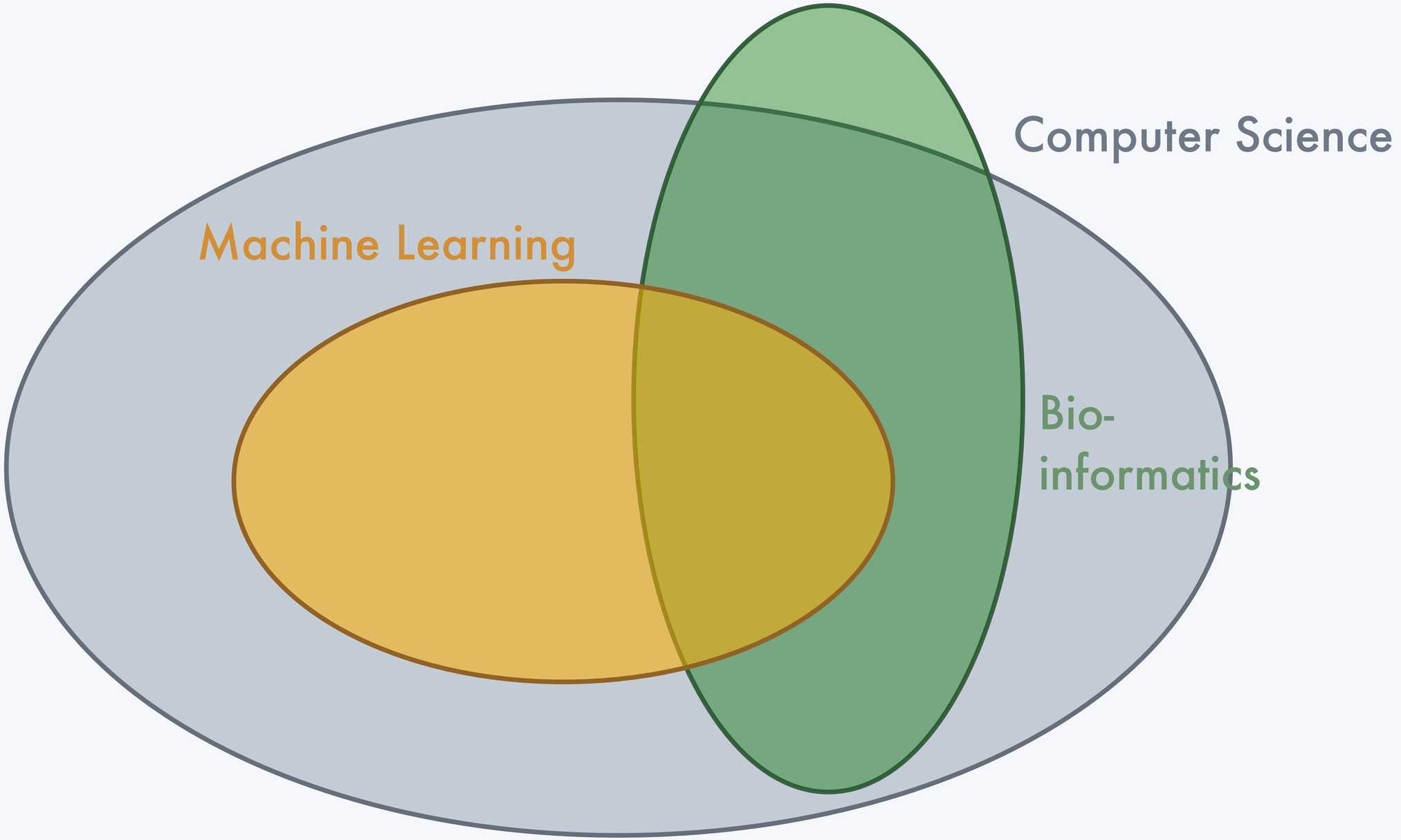
III) Perceptrons and first neural networks

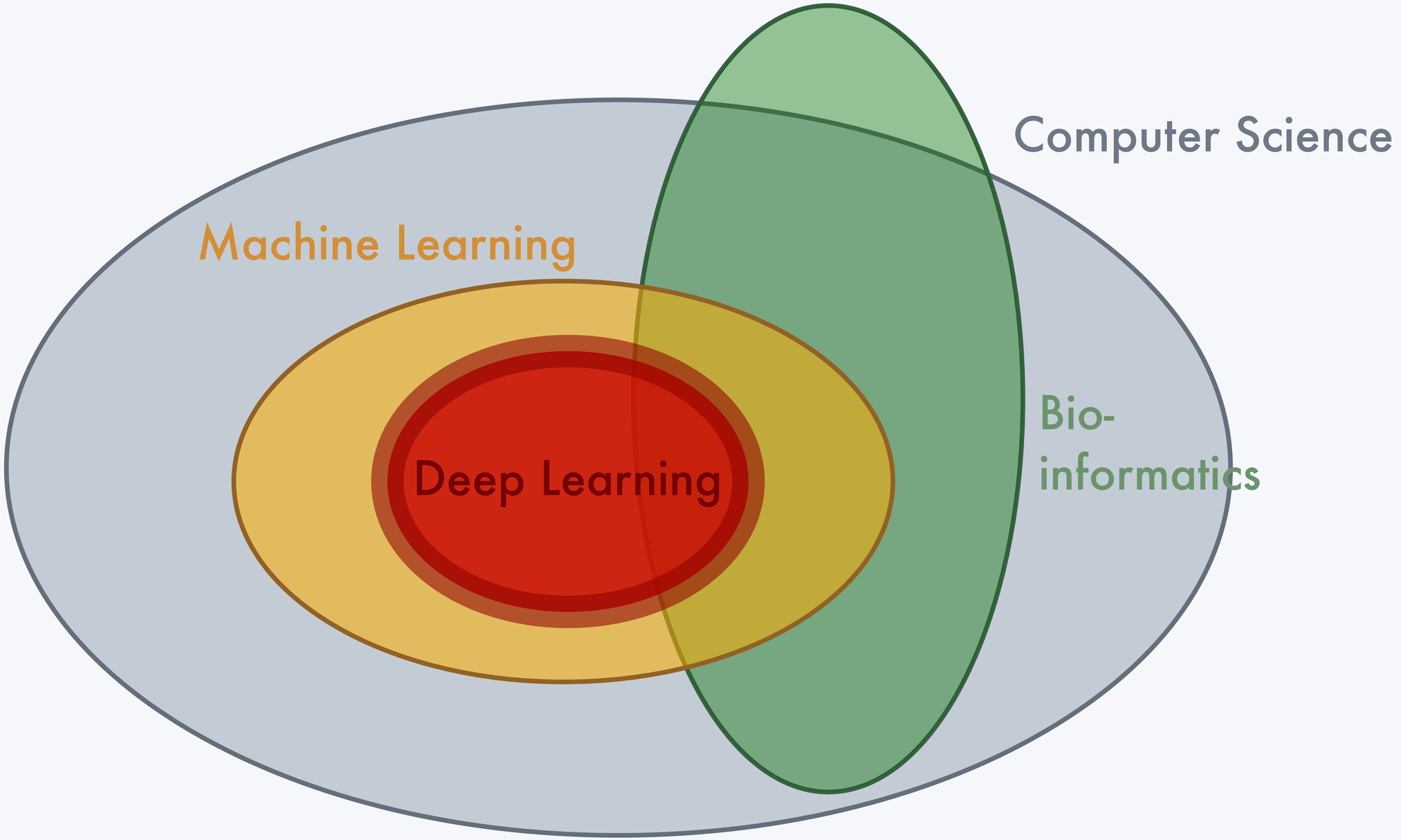
I) A bit of context and history




Computer Science


Computer Science
Bio-
informatics

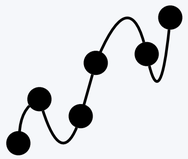
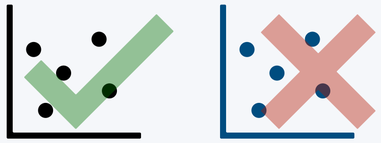

Computer Science
Machine Learning
Bio-
informatics


Computer Science
Machine Learning
Deep Learning
Bio-
informatics

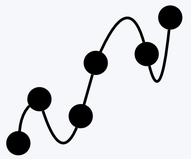
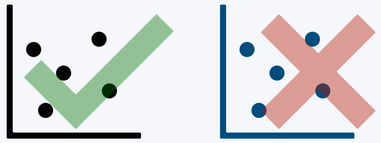
What can we do with deep learning:

- Do **better** what we already know how to do 
- Do **faster** what we already know how to do 
- Do things we **did not know** how to do 
- **Learn** things we don’t know 

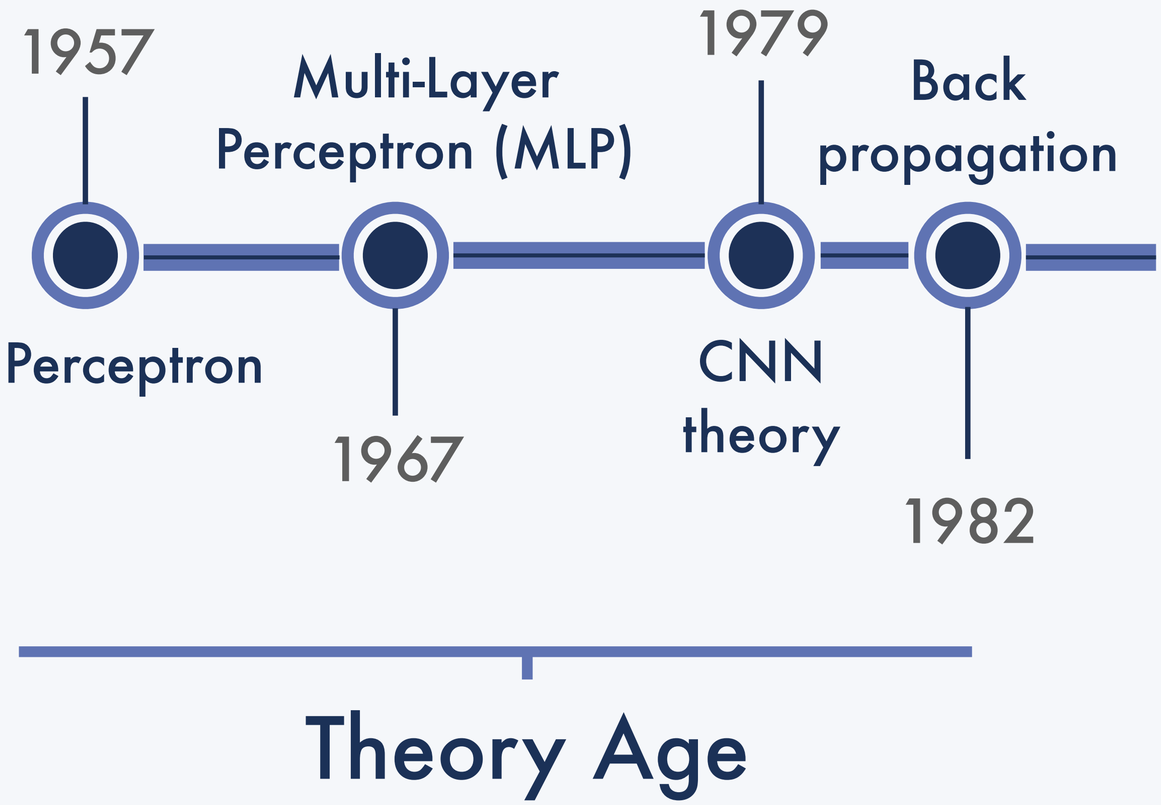
But what can we **also** do with deep learning:

- Overfit (learn by heart your data) 
- Fail to generalise 
- Hallucinate 

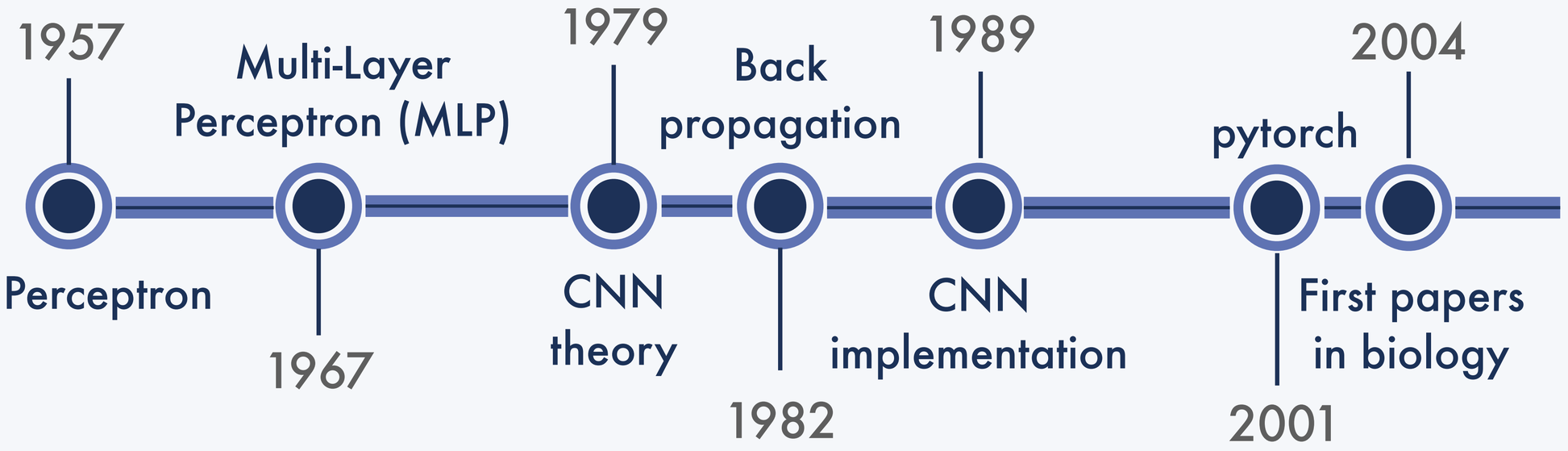
But what can we **also** do with deep learning:

- Overfit (learn by heart your data) 
- Fail to generalise 
- Hallucinate 
- Destroy the planet 

A brief history of deep learning



A brief history of deep learning

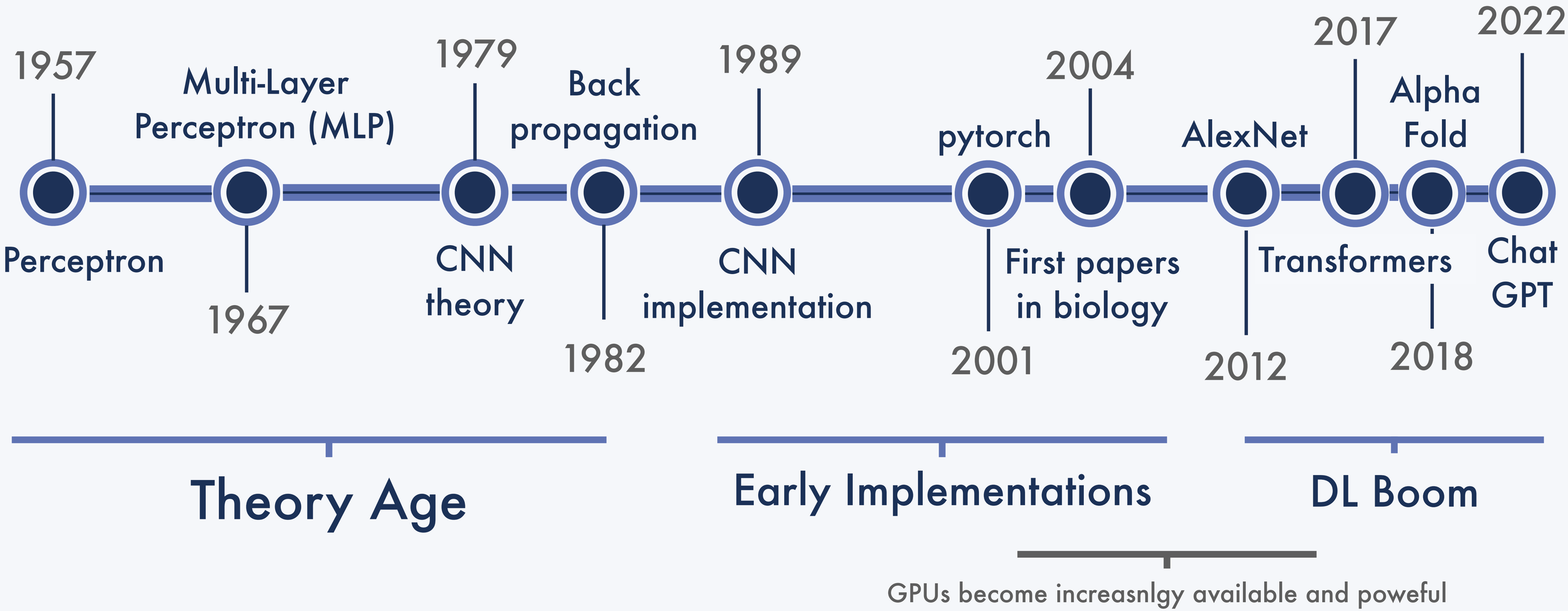


Theory Age

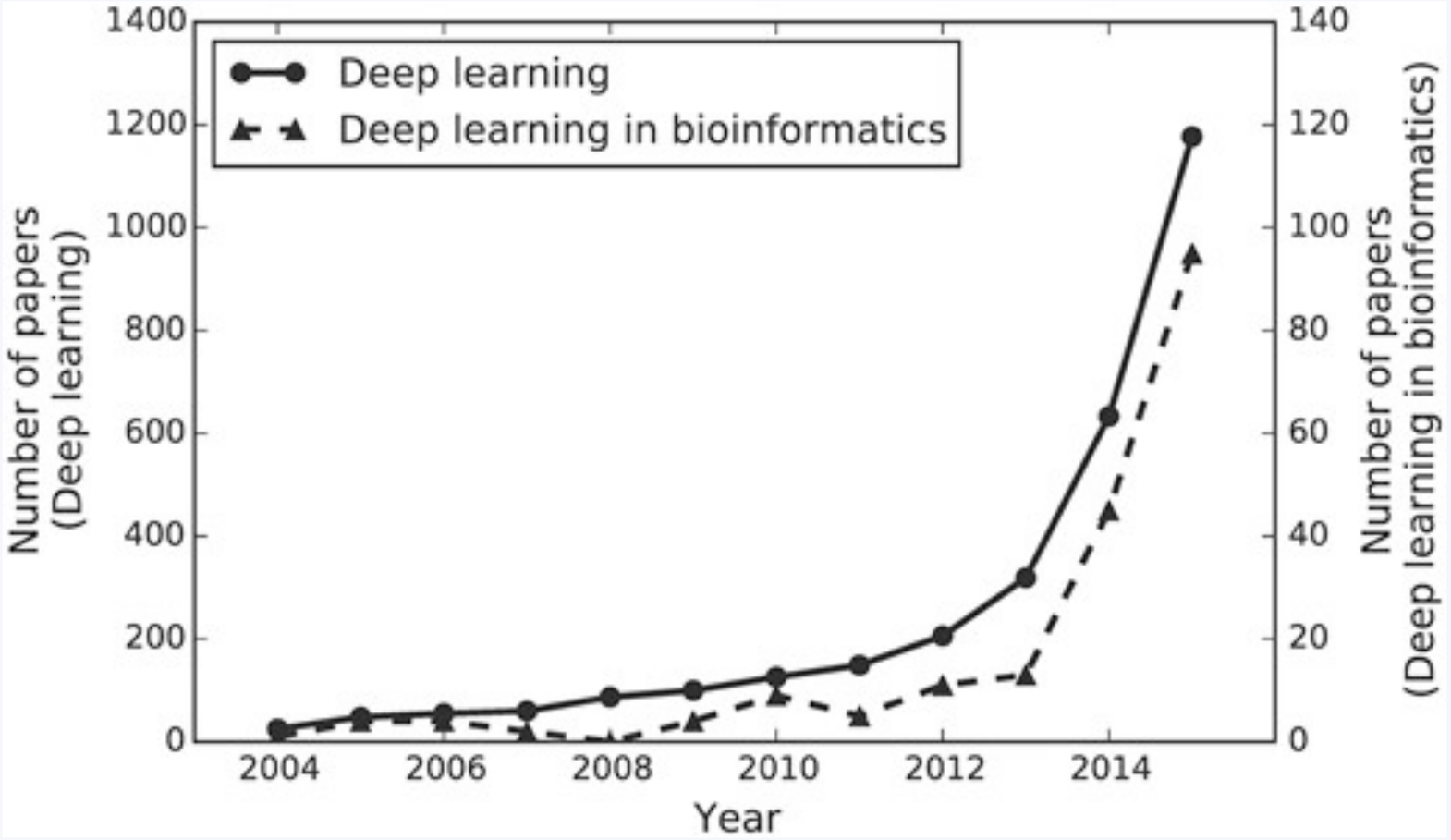
Early Implementations

GPUs become increasnlgy available and poweful

A brief history of deep learning



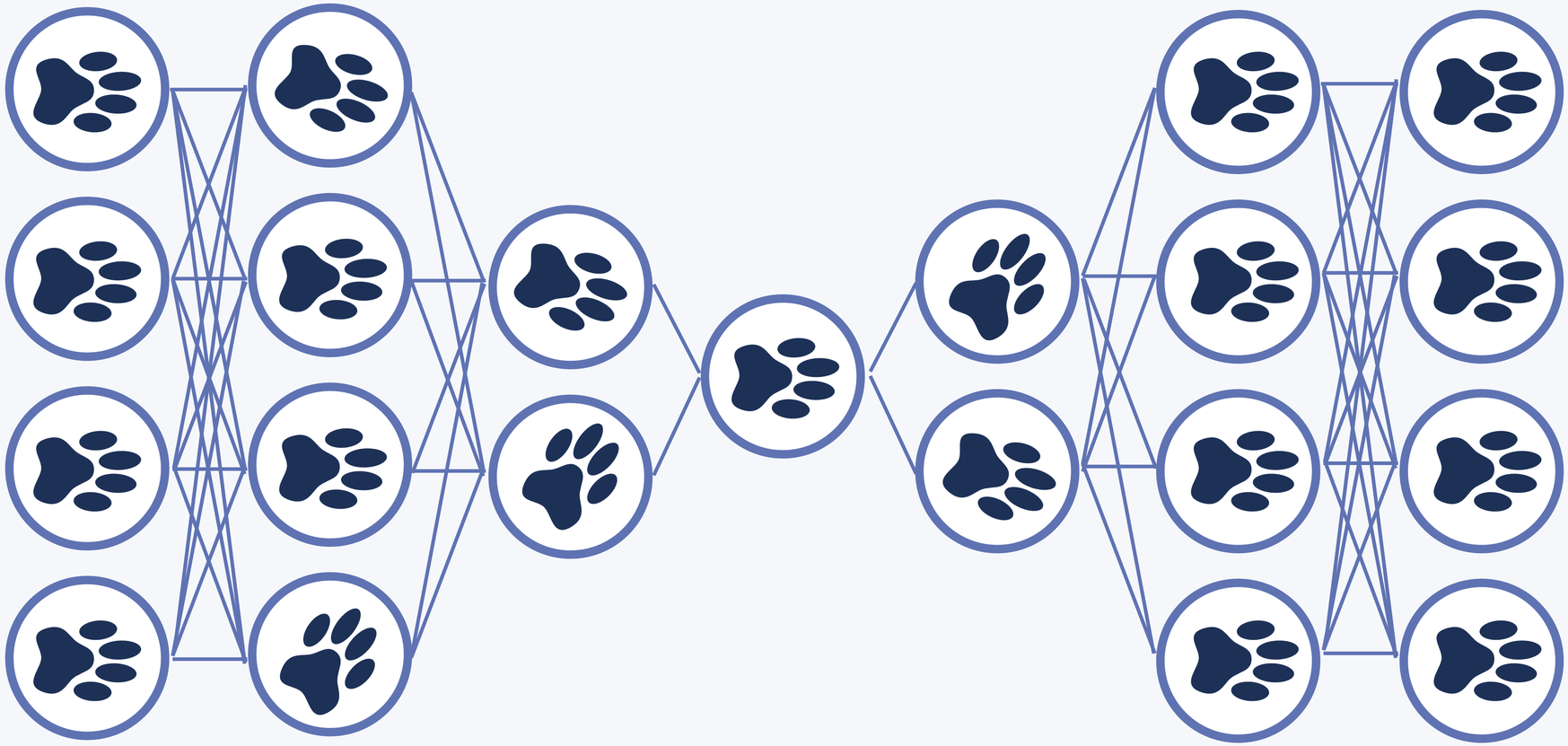
In bioinformatics



Approximate number of published deep learning articles by year. The number of articles is based on the search results on http://www.scopus.com with the two queries: 'Deep learning,' 'Deep learning' AND 'bio*'.

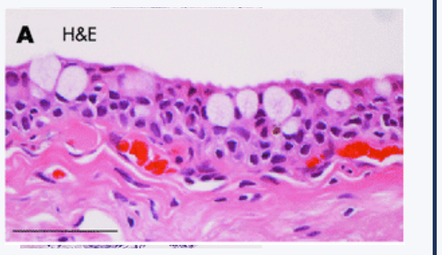
Deep learning in bioinformatics, Seonwoo Min , Byunghan Lee , Sungroh Yoon, *Briefings in Bioinformatics*, September 2017, Pages 851–869, https://doi.org/10.1093/bib/bbw068

II) The necessary steps of any deep learning algorithm



A. The basic idea of a deep learning model

You have data (**X**)



Whole Slide Image

T – T
T – C
G – G
A – A

Mutation status

Gene expression

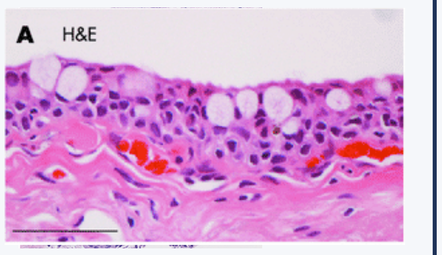
Molecule graph

	Age	Sex	BMI
0	45	M	22
1	70	F	18
2	81	F	30
3	23	M	24

Clinical data

A. The basic idea of a deep learning model

You have data (**X**)



Whole Slide Image

T — T
T — C
G — G
A — A

Mutation status

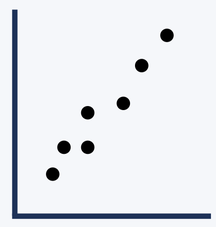
Gene expression

Molecule graph

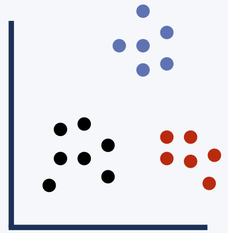
	Age	Sex	BMI
0	45	M	22
1	70	F	18
2	81	F	30
3	23	M	24

Clinical data

You want to predict something (**Y**) about this data



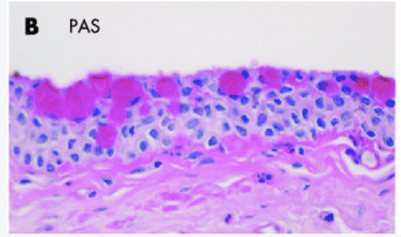
Prognosis



Cancer / Benign / Non Cancer

	Age	Sex	BMI
0	45	M	22
1	70	F	18
2	81	F	30
3	23	M	24

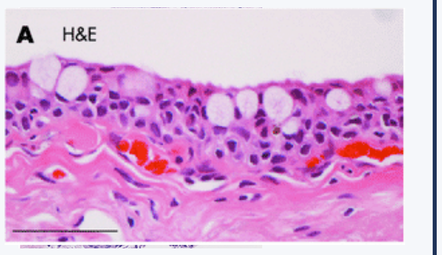
Clinical data



Whole Slide Image

A. The basic idea of a deep learning model

You have data (**X**)



Whole Slide Image

T — T
T — C
G — G
A — A

Mutation status

Gene expression

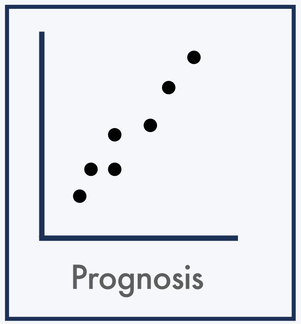
Molecule graph

	Age	Sex	BMI
0	45	M	22
1	70	F	18
2	81	F	30
3	23	M	24

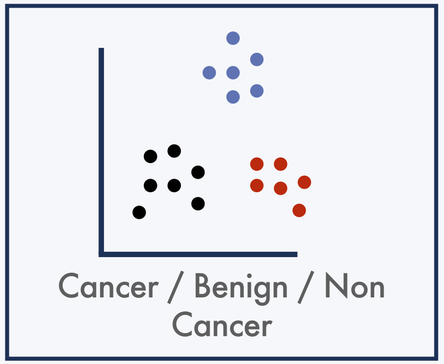
Clinical data

Define a function to go from X to Y

You want to predict something (**Y**) about this data



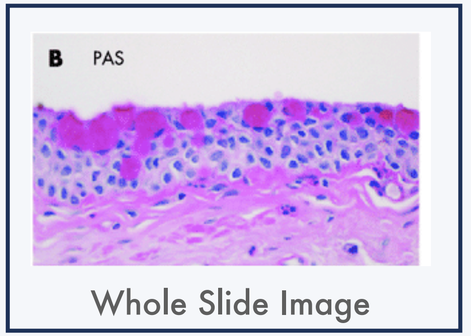
Prognosis



Cancer / Benign / Non Cancer

	Age	Sex	BMI
0	45	M	22
1	70	F	18
2	81	F	30
3	23	M	24

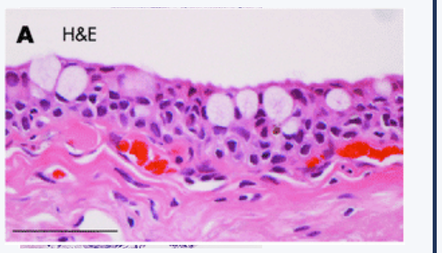
Clinical data



Whole Slide Image

A. The basic idea of a deep learning model

You have data (**X**)



Whole Slide Image

T — T
T — C
G — G
A — A
Mutation status

Gene expression

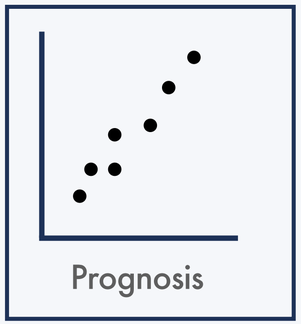

Molecule graph

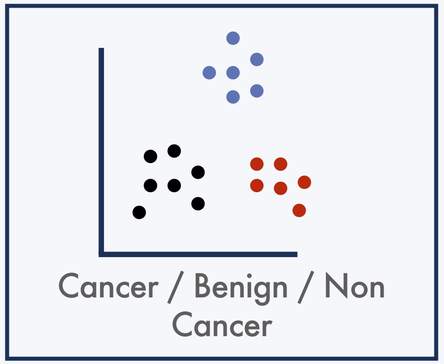
	Age	Sex	BMI
0	45	M	22
1	70	F	18
2	81	F	30
3	23	M	24

Clinical data

Define a DL model to go from X to Y

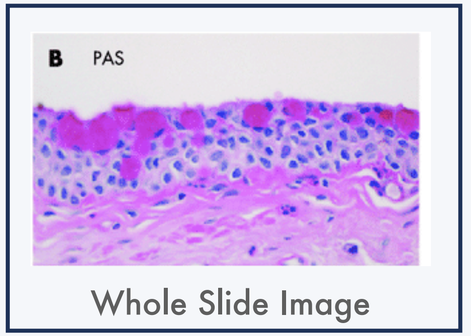
You want to predict something (**Y**) about this data


Prognosis


Cancer / Benign / Non Cancer

	Age	Sex	BMI
0	45	M	22
1	70	F	18
2	81	F	30
3	23	M	24

Clinical data



Whole Slide Image

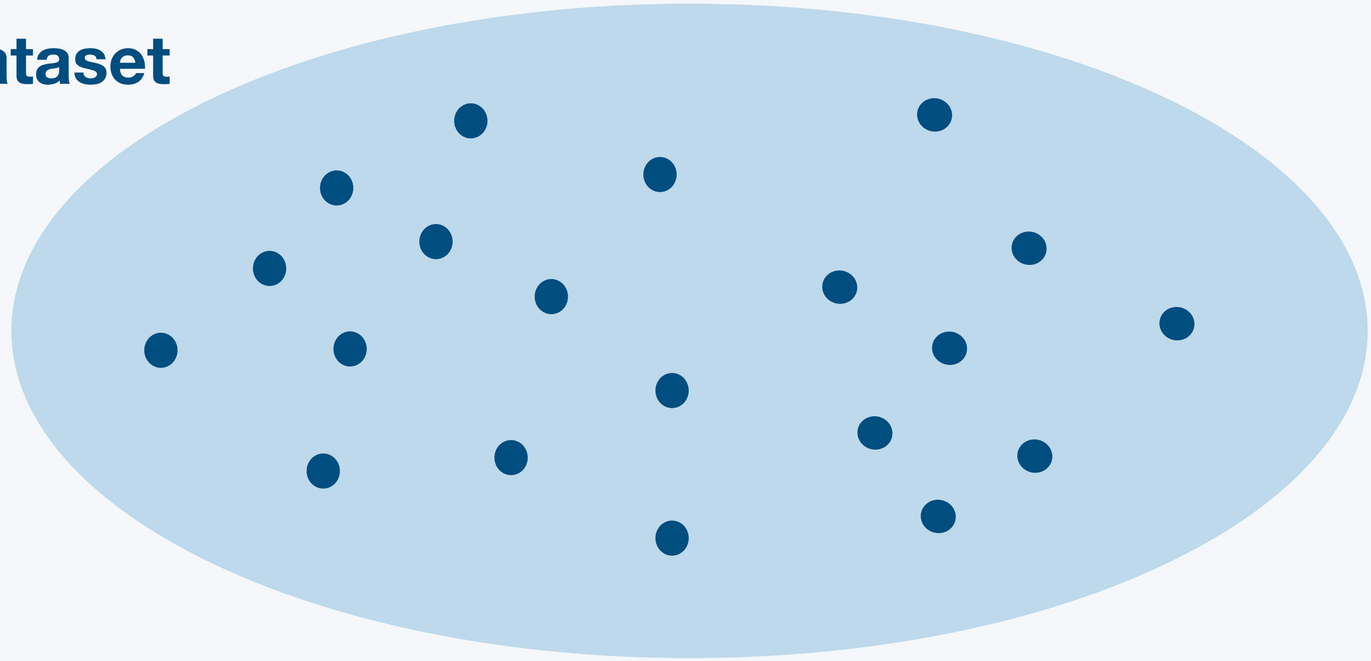
A. The basic idea of a deep learning model

Define a DL model (f)
from X to Y

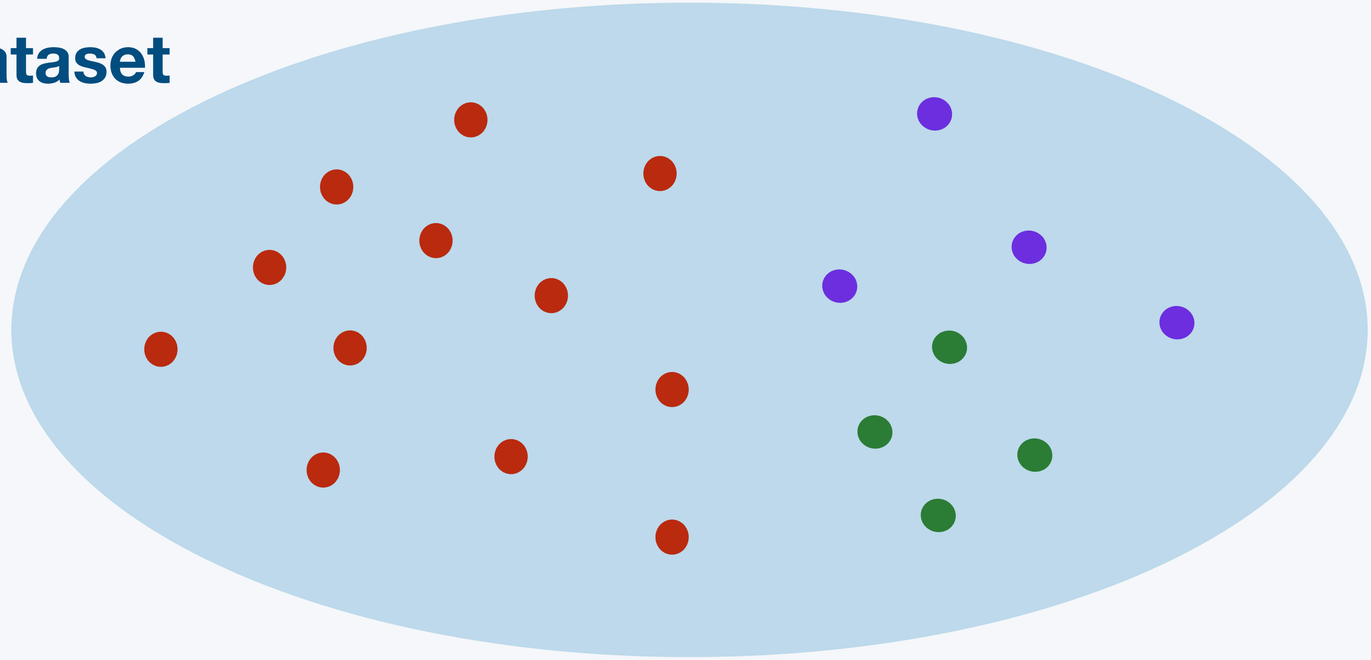
Define a way to tell your model when it is right or wrong, and how to **learn** from this information

$$\mathscr{L} = \text{prediction} - \text{target} \qquad f_{t+1} = f_t + \nabla \mathscr{L}_t$$

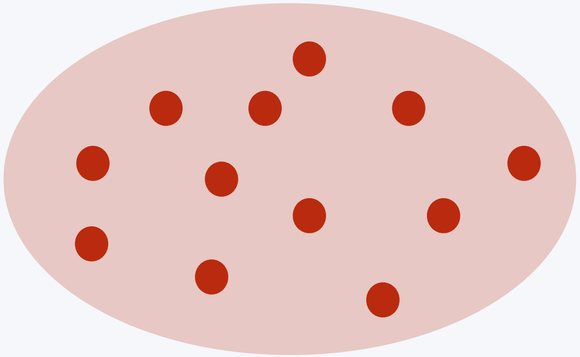
Full Dataset



Full Dataset

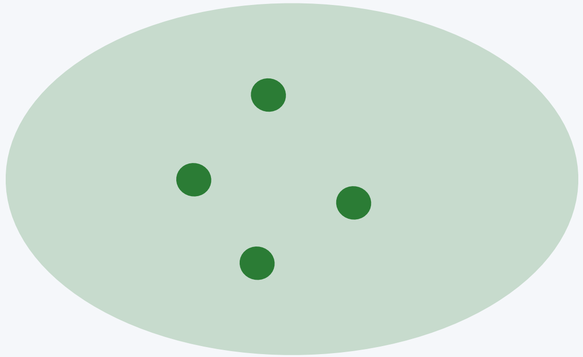


Training



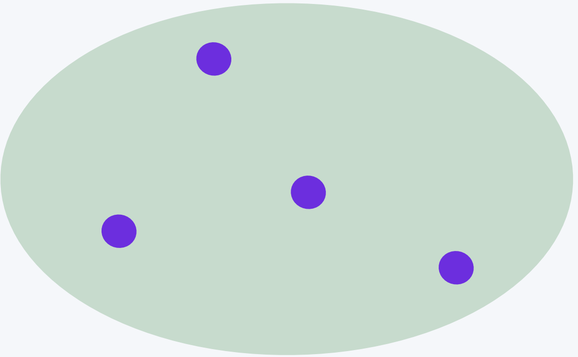
Fit your model

Validation



Make sure it
does not overfit

Testing



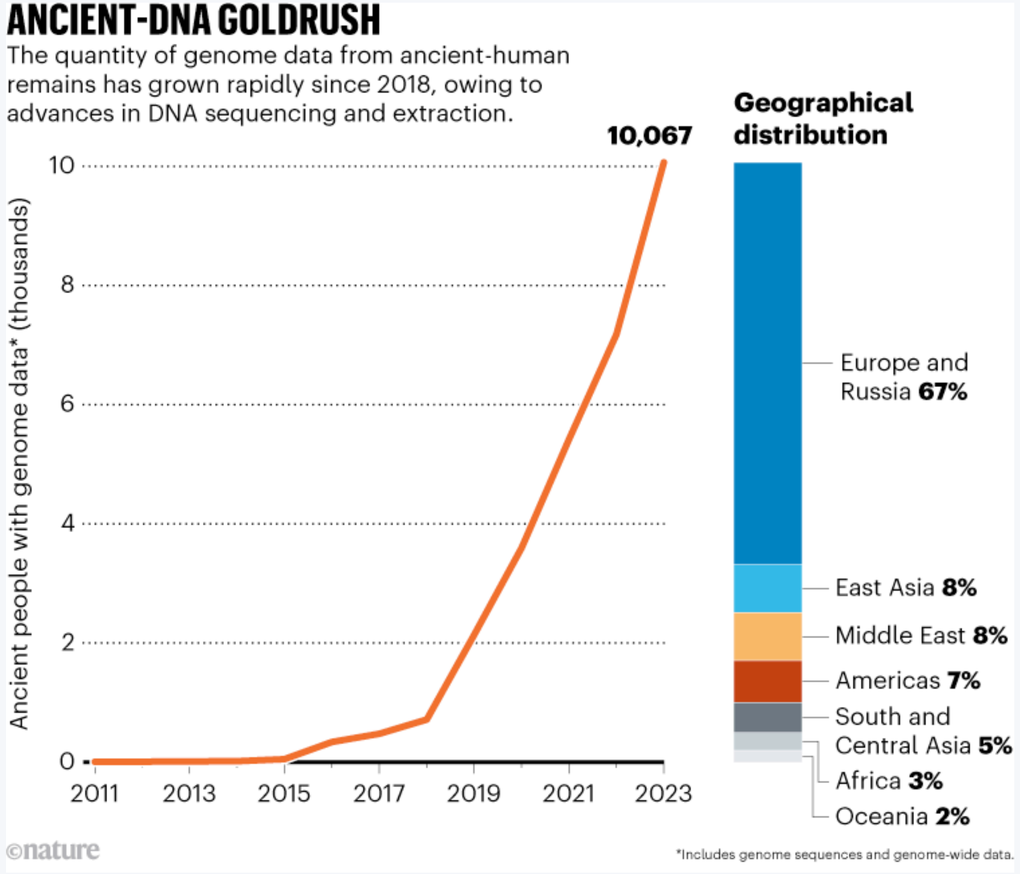
Check metrics on
(external) unseen data

Important concept

Ground Truth

For almost all deep learning models (see course 3 for exceptions), you need a **dataset already measured**, i.e. a dataset where you know **the truth Y** that you want to predict about X. We call this the **ground truth,** or **labels.**

This is usually a big bottleneck for deep learning applications, as you need a lot of data, which can be expensive or impossible to compute. But this is also why the deep learning era is growing: more and more data is available, diminishing this bottleneck step by step. This is, for example, very clear in spatial transcriptomic or single-cell genomics.



https://www.nature.com/articles/d41586-023-01403-4

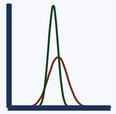
B. **The** deep learning algorithm

B. **The** deep learning algorithm

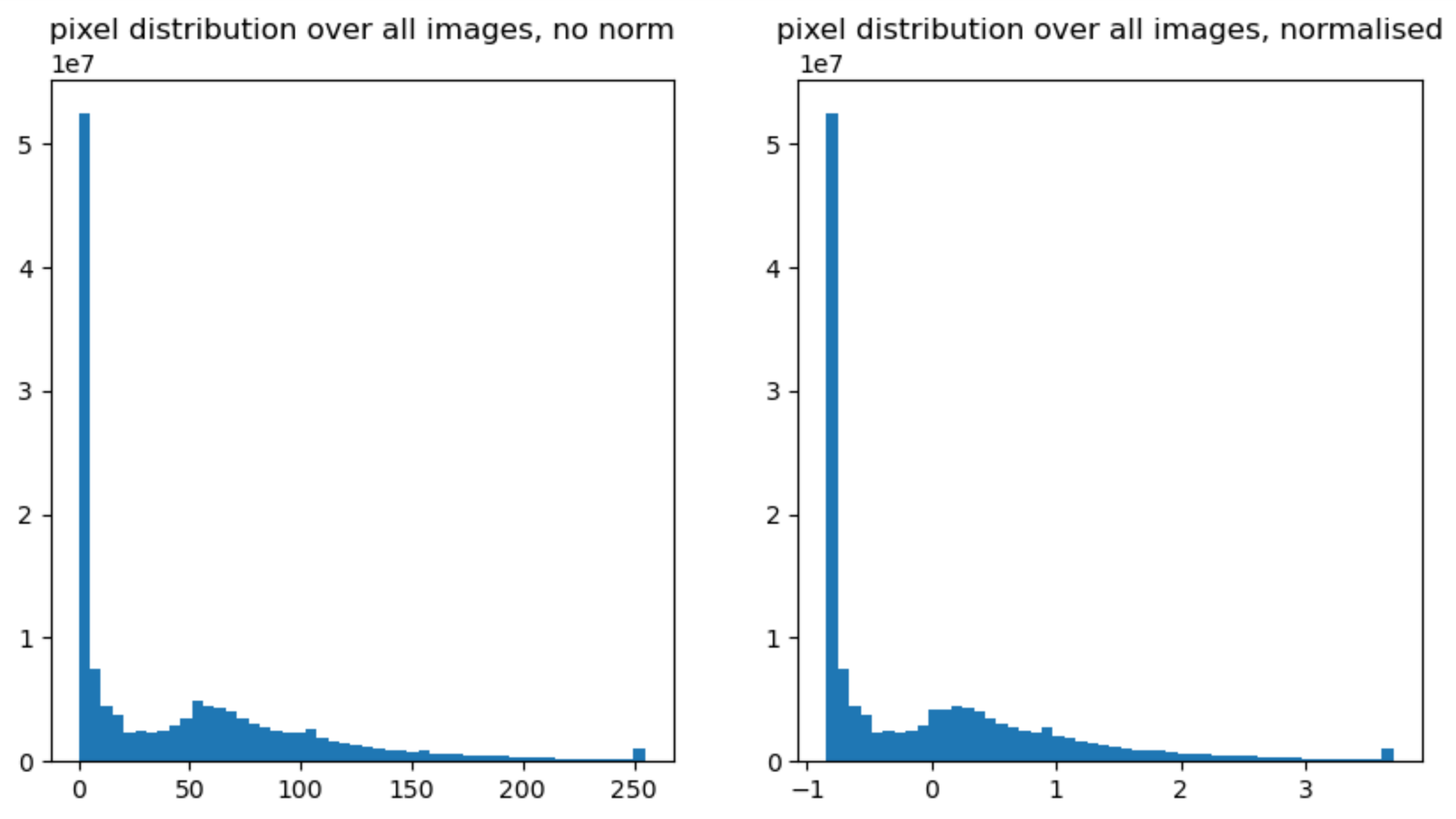
1- Define your data: X and Y

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

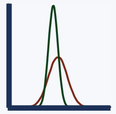
Deep learning algorithms love Gaussian-like data: try to normalise it this way!



B. **The** deep learning algorithm

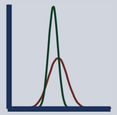
1- Define your data: X and Y

2- Preprocess your data



B. **The** deep learning algorithm

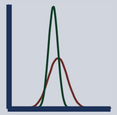
1- Define your data: X and Y 

2- Preprocess your data 

Data collection

B. **The** deep learning algorithm

1- Define your data: X and Y 

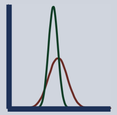
2- Preprocess your data 

3- Define your model architecture 

Data collection

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

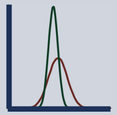
3- Define your model architecture 

4- Define your loss function 

Data collection

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

Data collection

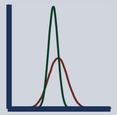
3- Define your model architecture 

4- Define your loss function 

Model definition

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

Data collection

3- Define your model architecture 

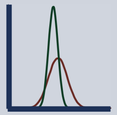
4- Define your loss function 

Model definition

5- Predict $\hat{Y}$ from X (untrained) 

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

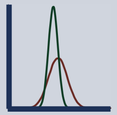
6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

Data collection

Model definition

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

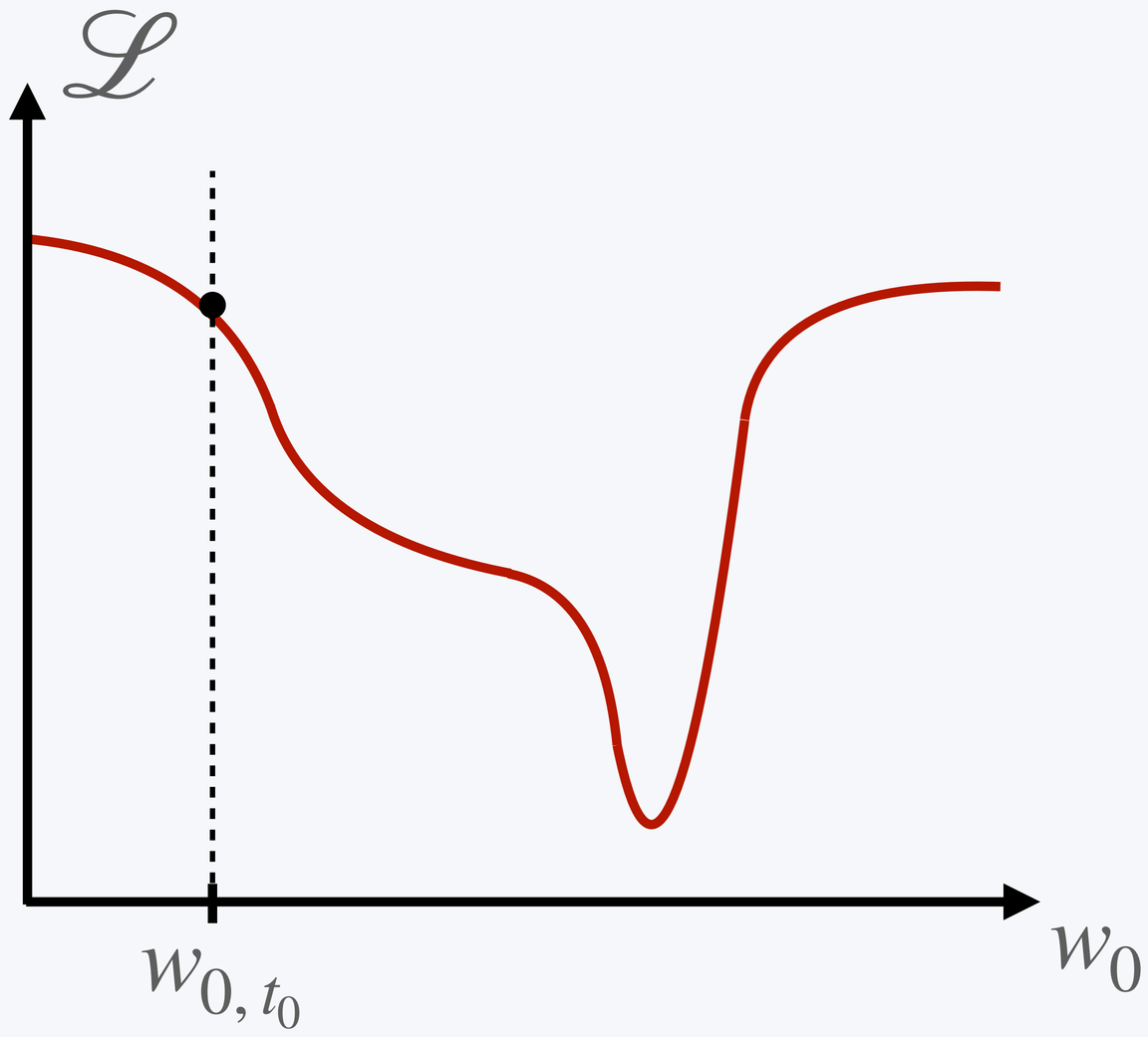
6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

Data collection

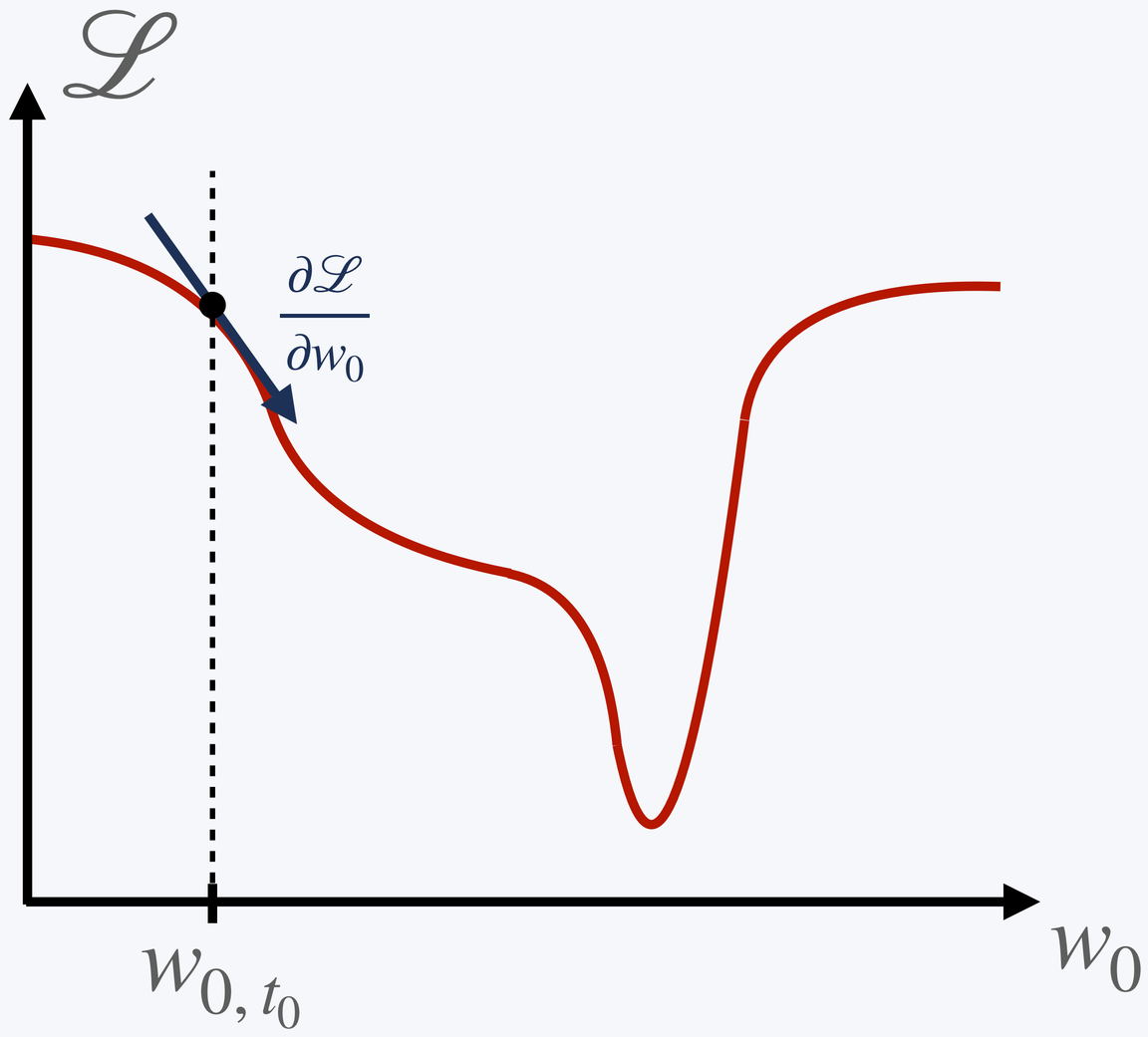
Model definition

We optimise the weights of our model with gradient descent



$$w_{0,t_1} = w_{0,t_0} +$$

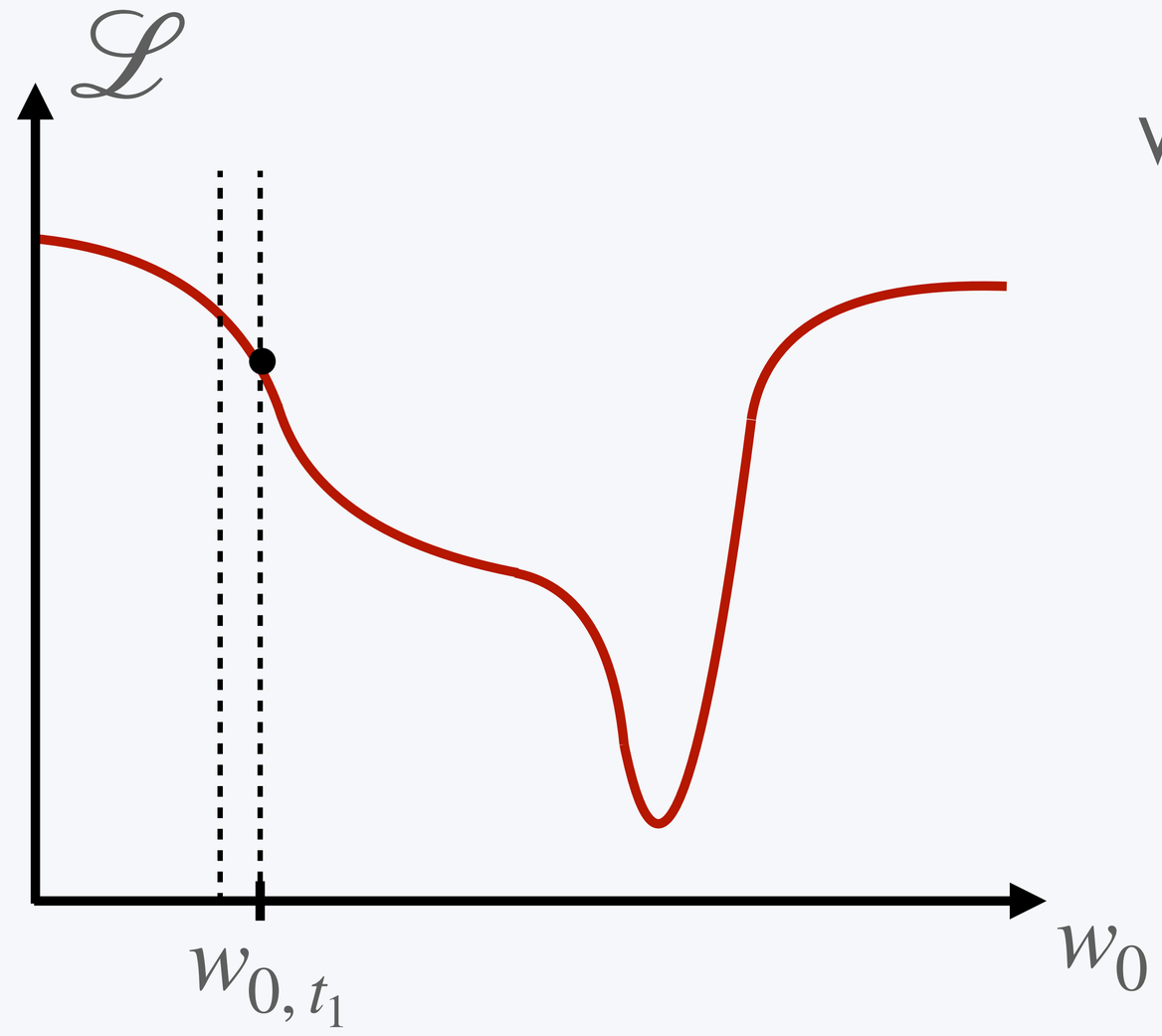
We optimise the weights of our model with gradient descent



The derivative gives you the steepest slope towards the minimum of your loss!

$$w_{0,t_1} = w_{0,t_0} +$$

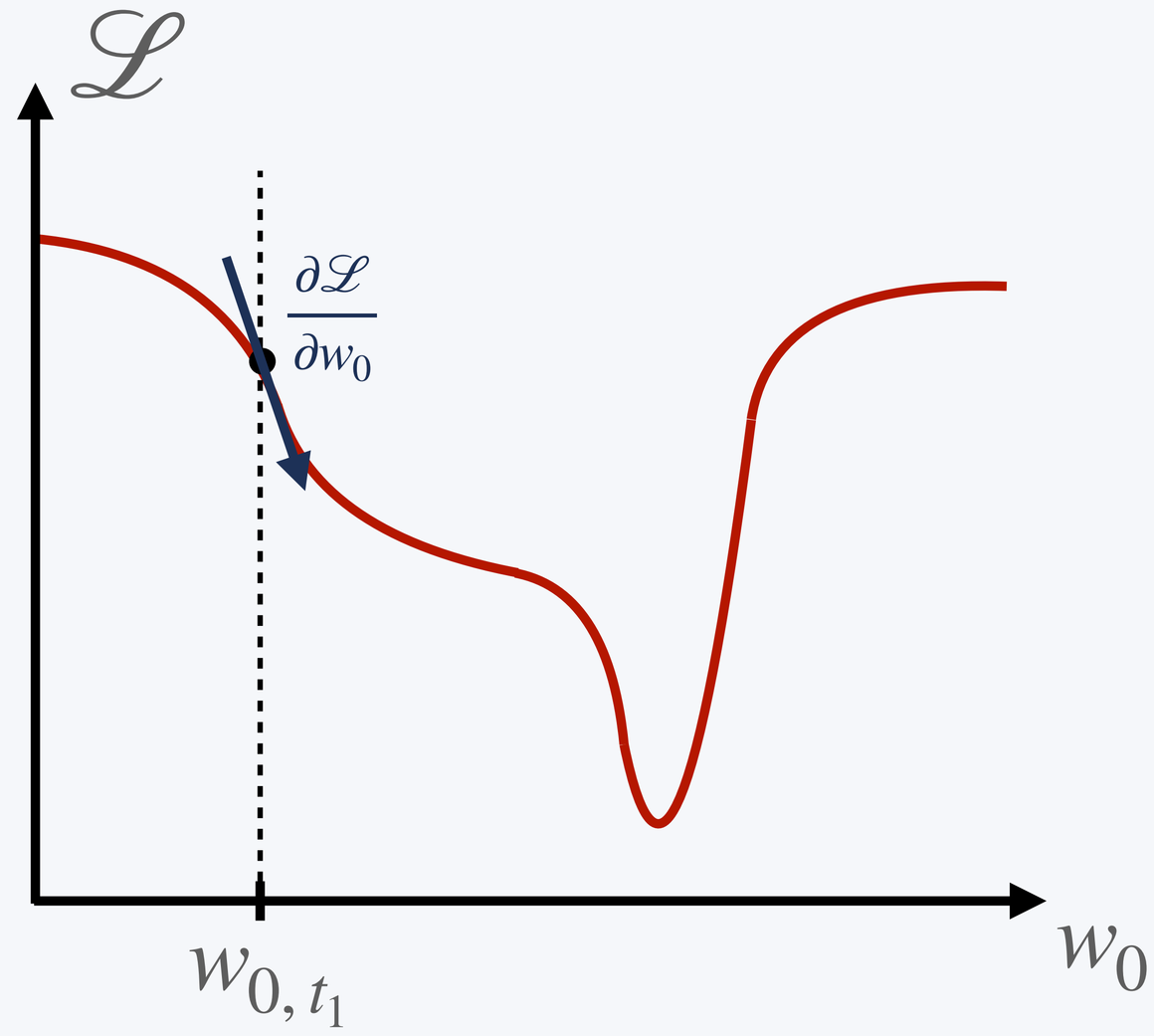
We optimise the weights of our model with gradient descent



We update our weight by moving it towards the derivative!

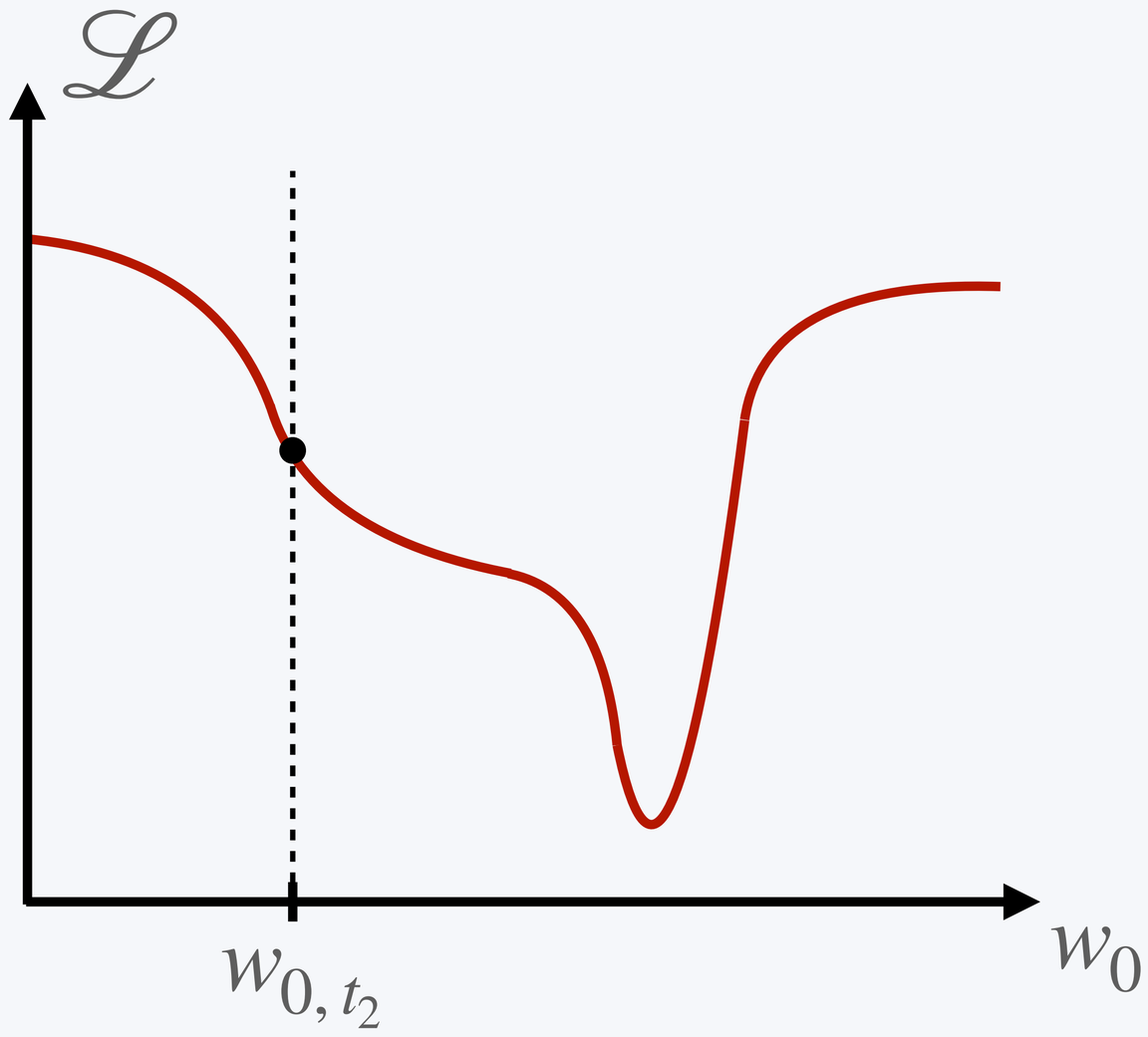
$$w_{0,t_1} = w_{0,t_0} + \frac{\partial \mathscr{L}}{\partial w_0}$$

We optimise the weights of our model with gradient descent



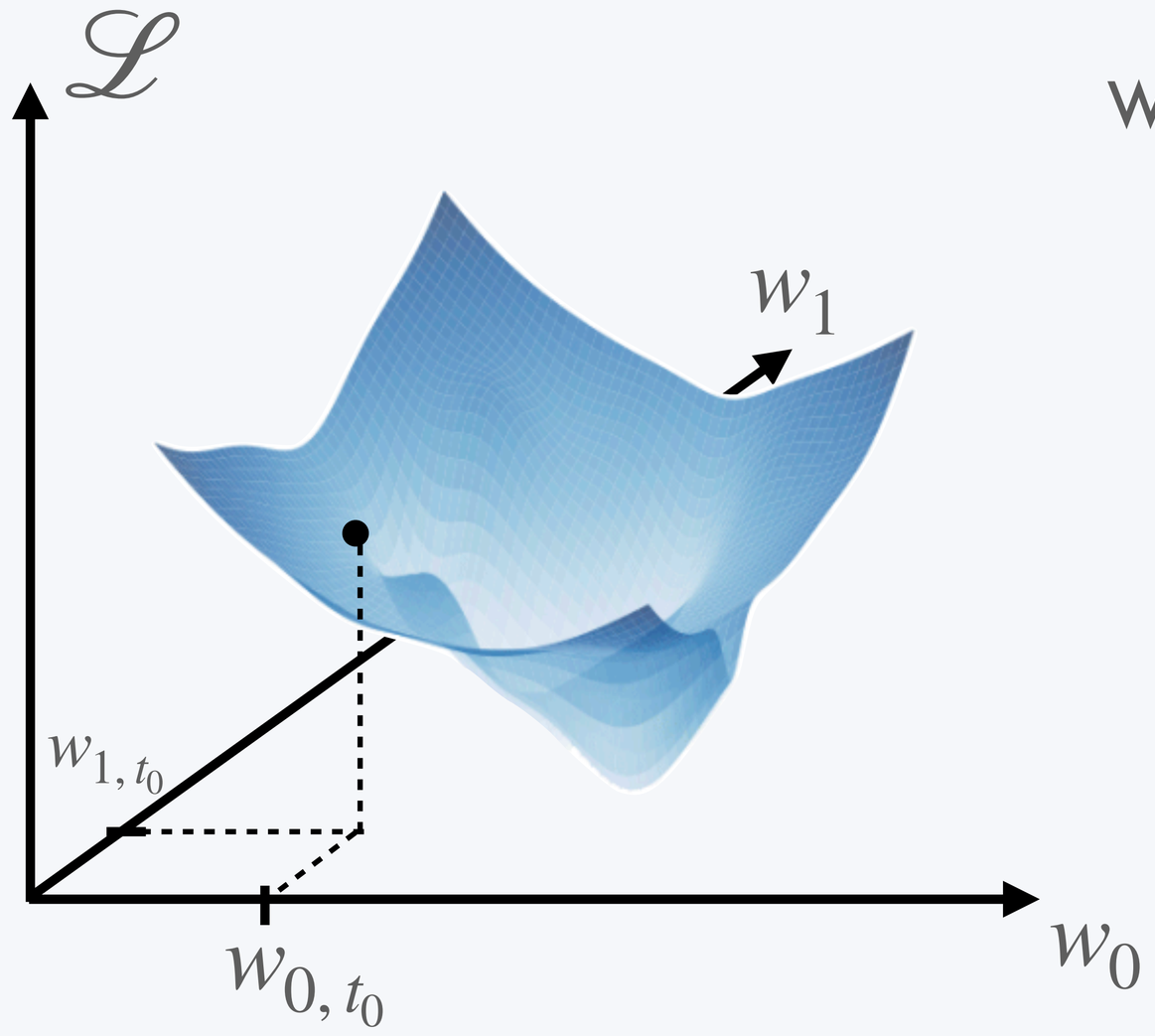
$$w_{0,t_1} = w_{0,t_0} + \frac{\partial \mathscr{L}}{\partial w_0}$$

We optimise the weights of our model with gradient descent



$$w_{0,t_2} = w_{0,t_1} + \frac{\partial \mathscr{L}}{\partial w_0}$$

We optimise the weights of our model with gradient descent

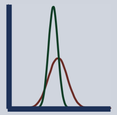


When we have more than one parameter, we use the gradient instead of the derivative, each parameter updated in its direction.

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t + \nabla \mathscr{L}$$

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function 

Model definition

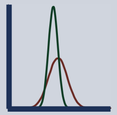
5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function 

Model definition

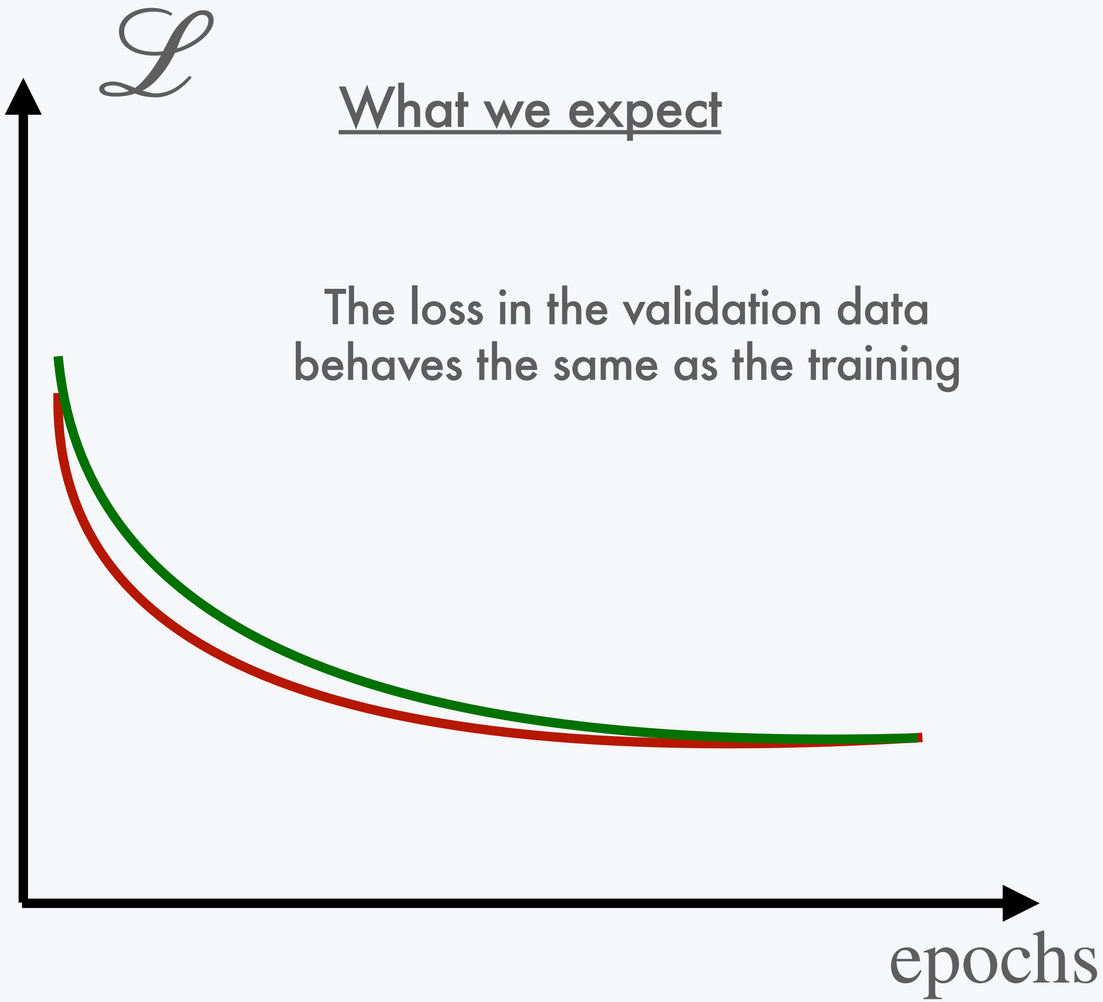
5- Predict $\hat{Y}$ from X (untrained) 

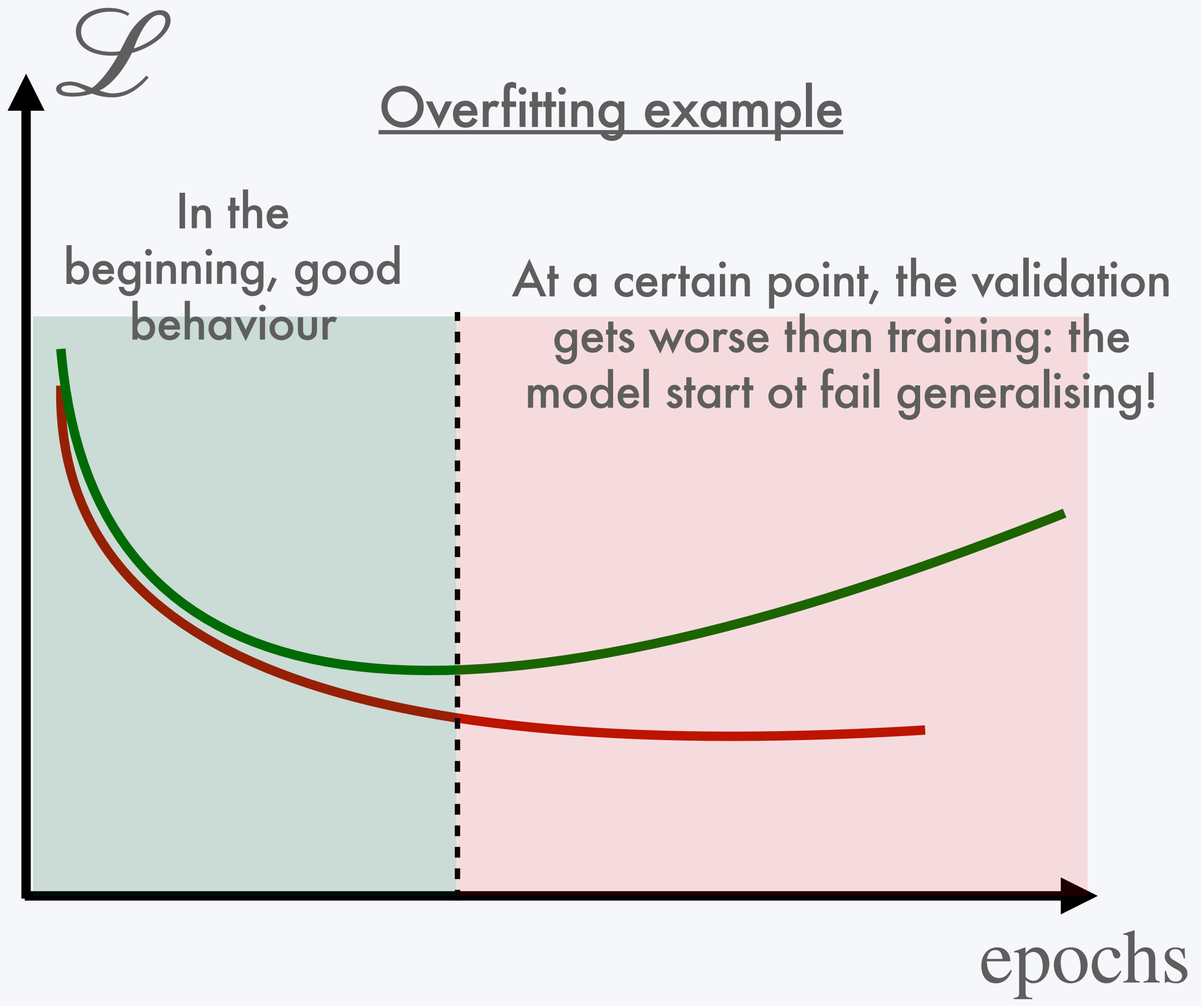
6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

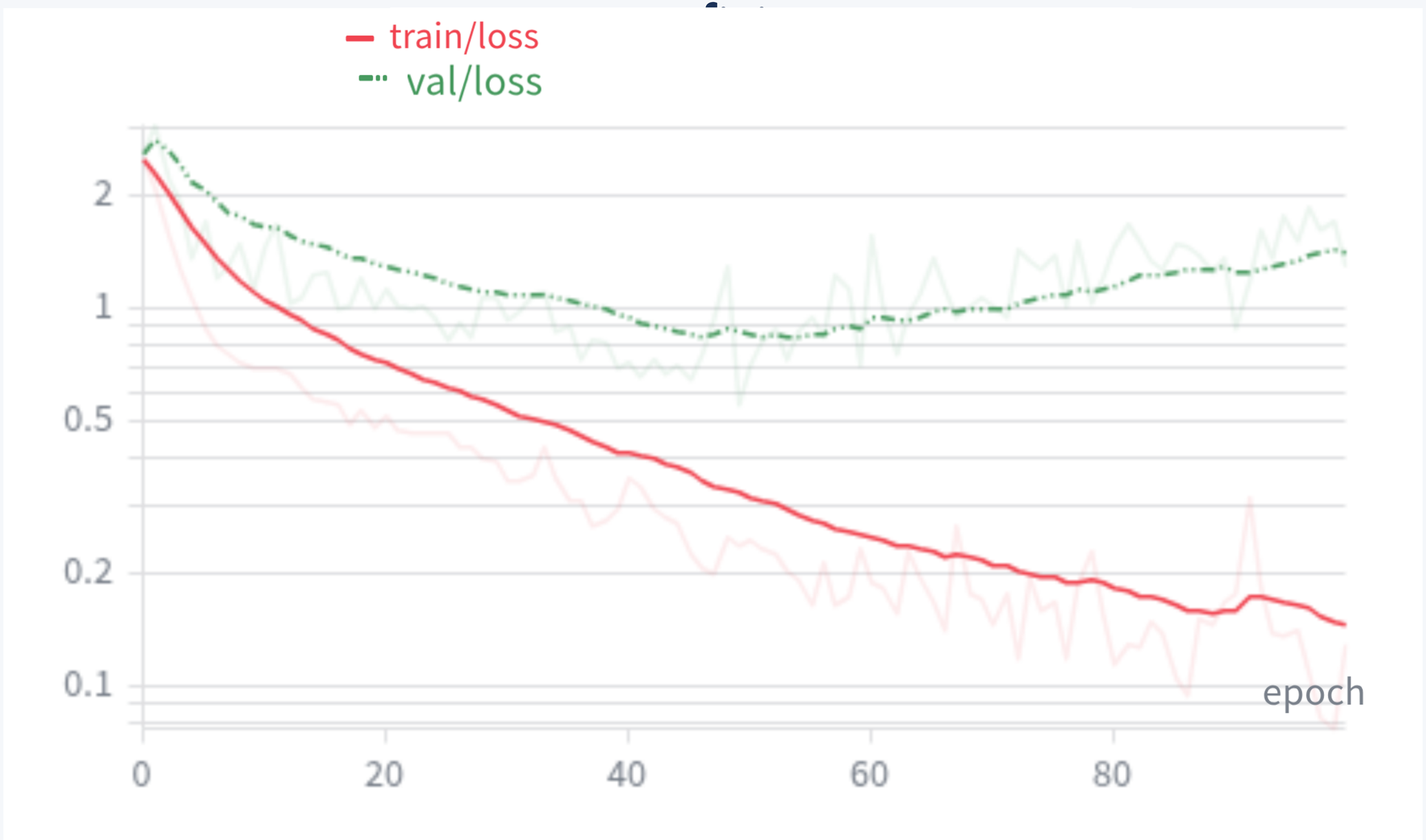
8- Do 5 and 6 in Validation ✔

We use validation to see when our model starts overfitting



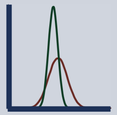


We use validation to see when our model starts



B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function 

Model definition

5- Predict $\hat{Y}$ from X (untrained) 

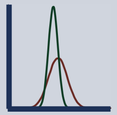
6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function 

Model definition

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

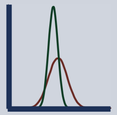
7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting 🔄

B. **The** deep learning algorithm

1- Define your data: X and Y 

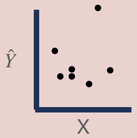
2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function 

Model definition

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

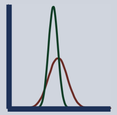
8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting 🔄

Training

B. **The** deep learning algorithm

1- Define your data: X and Y 

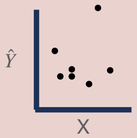
2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function 

Model definition

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

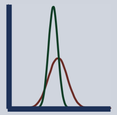
9- Loop on 5 to 8 untill convergence wo overfitting 🔄

Training

10- Test on unseen data 

B. **The** deep learning algorithm

1- Define your data: X and Y 

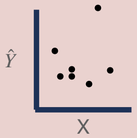
2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function 

Model definition

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

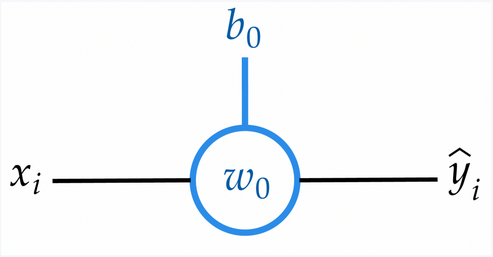
9- Loop on 5 to 8 untill convergence wo overfitting 🔄

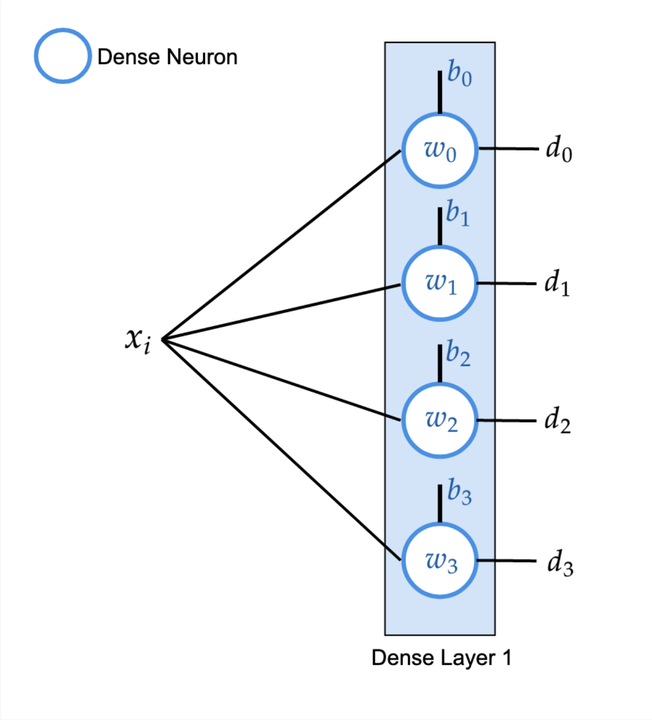
Training

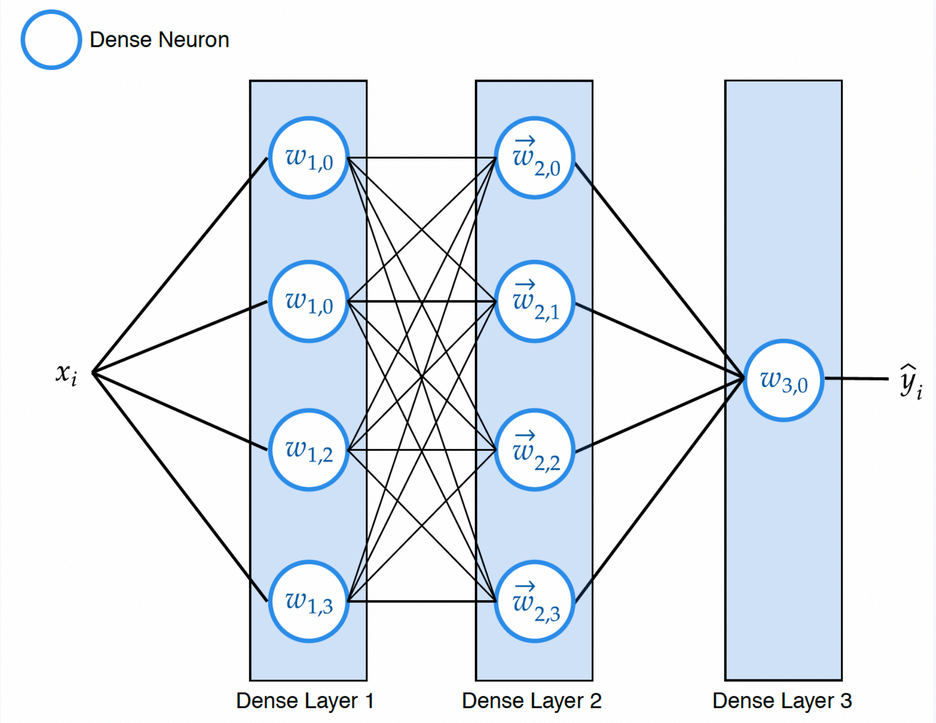
10- Test on unseen data 

Testing

III) Perceptrons and first neural networks

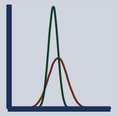






B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

Data collection

3- Define your model architecture 

4- Define your loss function

Model

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation 

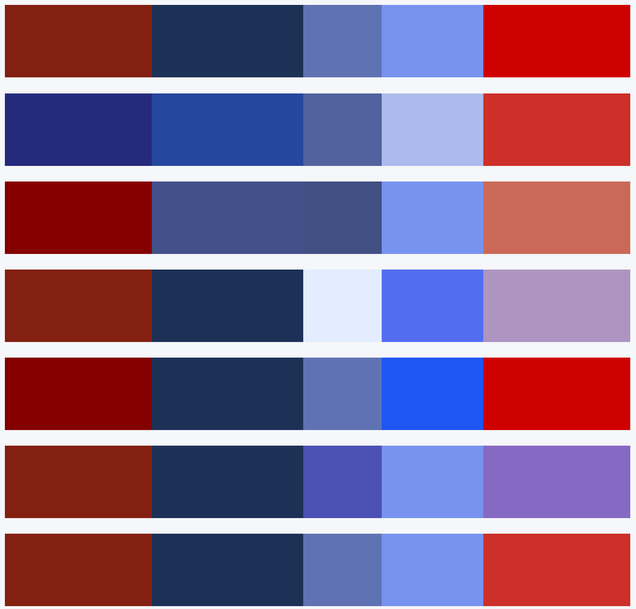
9- Loop on 5 to 8 untill convergence wo overfitting 

Training

10- Test on unseen data 

Testing

X



Gene expression

f(X) = Y

Y

Low risk

Low risk

High risk

Low risk

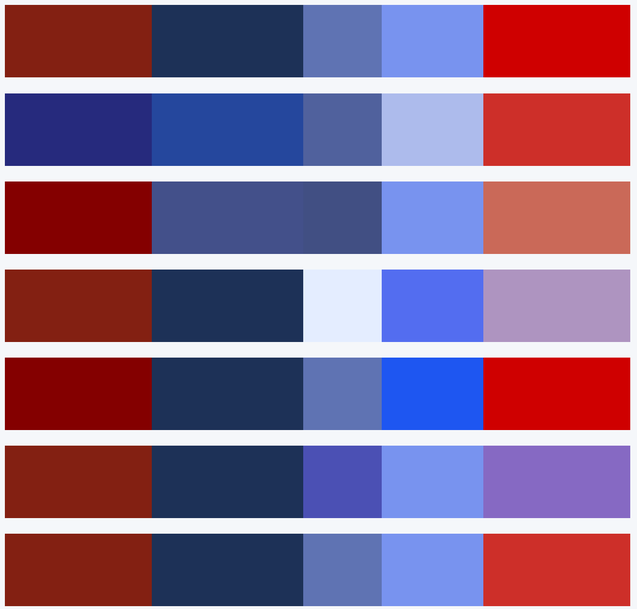
High risk

High risk

High risk

Prognostic

X



Gene expression

model

f(X) = Y

Y

Low risk

Low risk

High risk

Low risk

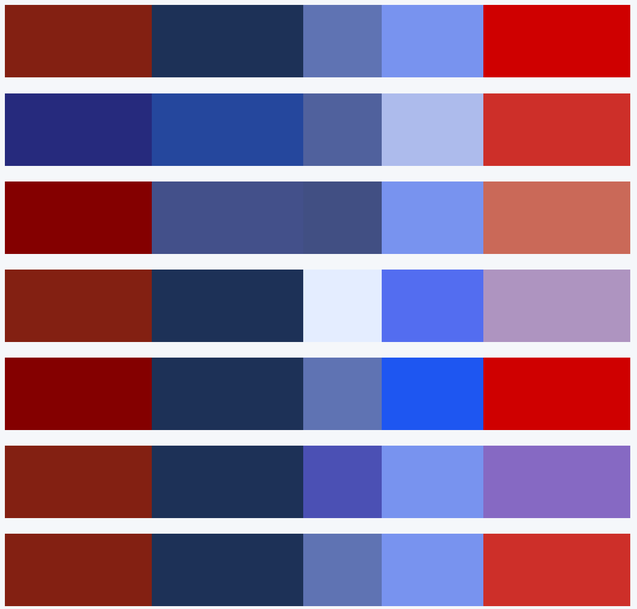
High risk

High risk

High risk

Prognostic

X



Gene expression

parameters
(or weights)

model

f_w(X) = Y

Y

Low risk
Low risk
High risk
Low risk
High risk
High risk
High risk

Prognostic

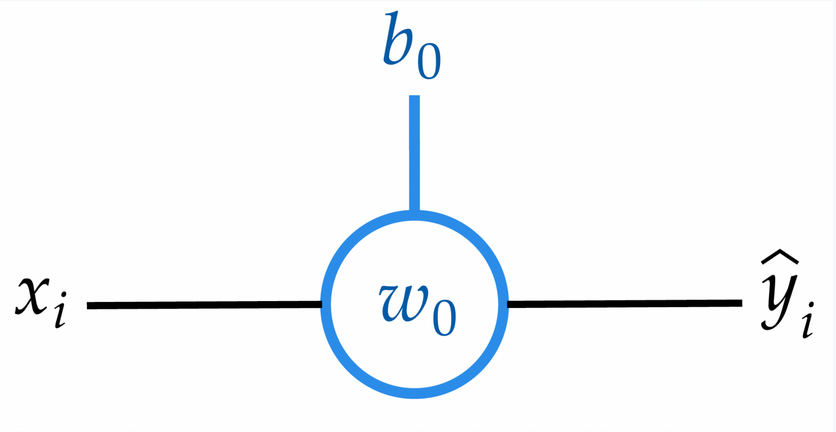
First example: the perceptron

$$f(x) = y$$

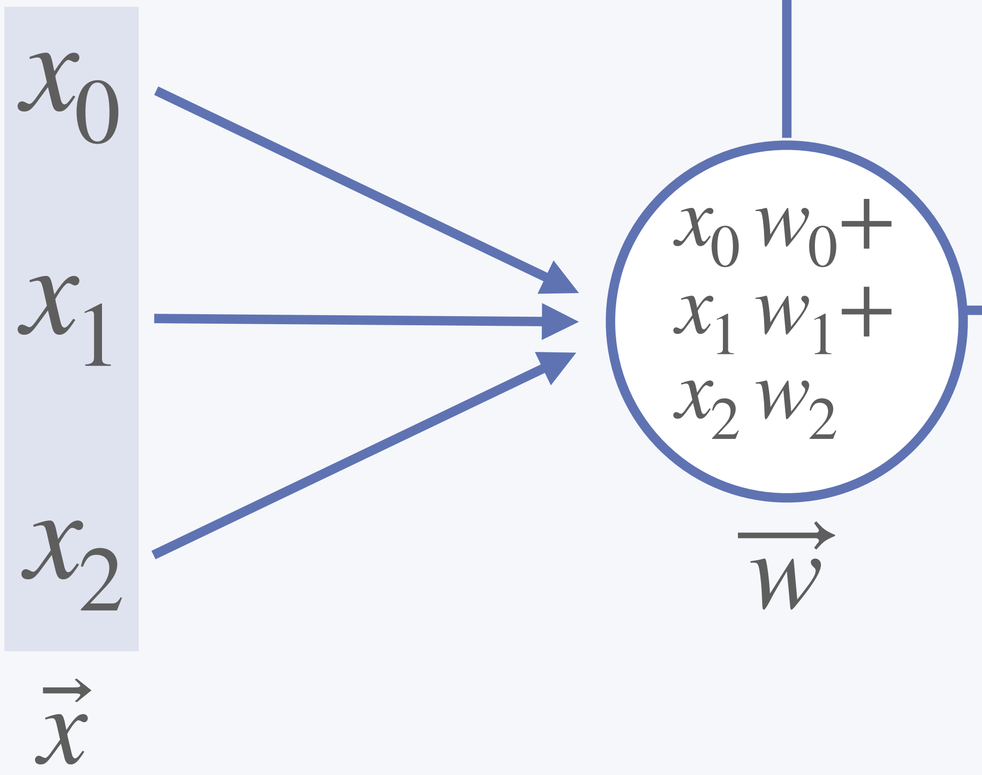
$$f(x) = a \times x + b$$

$$f(x) = w_0 \times x + b_0$$

(not exactly yet a perceptron)



What happens when you have more than one feature as input:



As many weights per neuron as input features!

$$\hat{y} = w_0\,x_0 + w_1 x_1 + w_2 x_2 + b$$

$$\hat{y} = \vec{x}\,.\,\vec{w} + b$$

This is similar to a logistic regression!

but b is the intercept, not the error

Neuron output: $$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2 + b_0$$

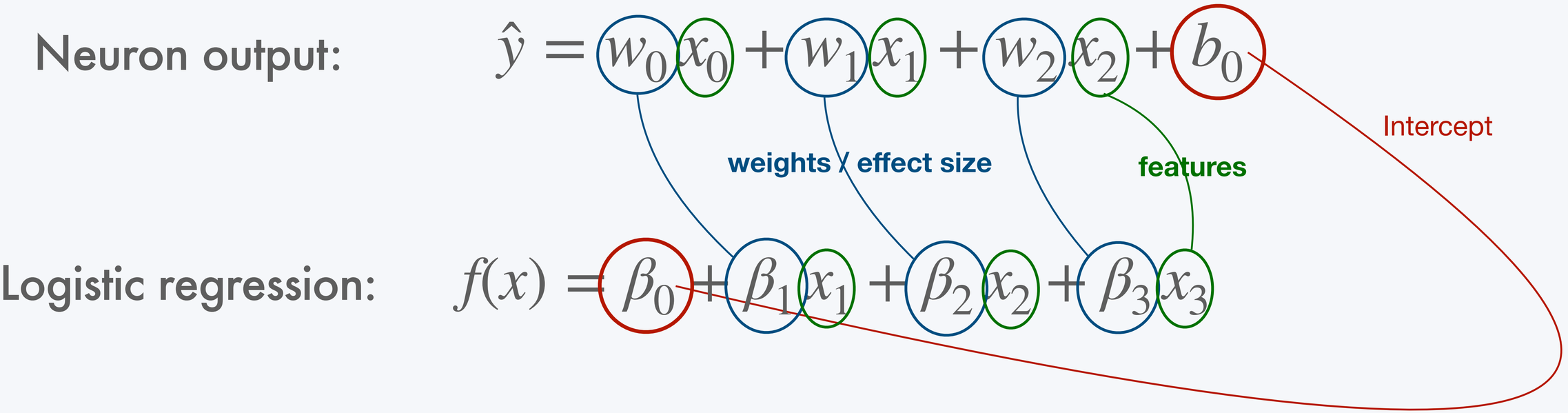
Logistic regression: $$f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

This is similar to a logistic regression!

but b is the intercept, not the error

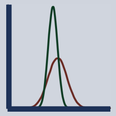
Neuron output: $\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2 + b_0$

Logistic regression: $f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$



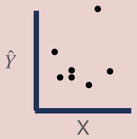
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

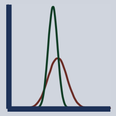
8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

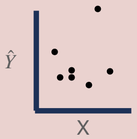
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

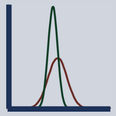
9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

X = age, Y = prognostic (high=1 or low=0)

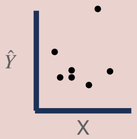
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

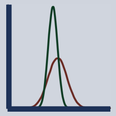
10- Test on unseen data 

X = age, Y = prognostic (high=1 or low=0)

X = X / 100, Y = Y

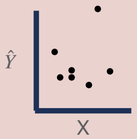
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

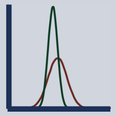
X = age, Y = prognostic (high=1 or low=0)

X = X / 100, Y = Y

$$f(x) = \hat{y} = w_0 x + b_0$$

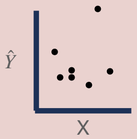
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

X = age, Y = prognostic (high=1 or low=0)

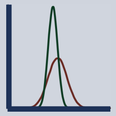
X = X / 100, Y = Y

$$f(x) = \hat{y} = w_0 x + b_0$$

$$\mathscr{L} = |\hat{y} - y|^2$$

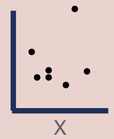
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

X = age, Y = prognostic (high=1 or low=0)

X = X / 100, Y = Y

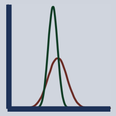
$$f(x) = \hat{y} = w_0 x + b_0$$

$$\mathscr{L} = |\hat{y} - y|^2$$

$$x = 0.85,\ w_{0,t_0} = 0.1,\ b_{0,t_0} = 0.3\ ;\ \hat{y} = 0.385$$

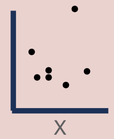
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

X = age, Y = prognostic (high=1 or low=0)

X = X / 100, Y = Y

$$f(x) = \hat{y} = w_0 x + b_0$$

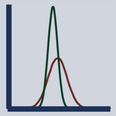
$$\mathscr{L} = |\hat{y} - y|^2$$

$$x = 0.85,\ w_{0,t_0} = 0.1,\ b_{0,t_0} = 0.3\ ;\ \hat{y} = 0.38$$

$$y = 1;\ \hat{y} = 0.38\ ;\ \mathscr{L} = 0.92$$

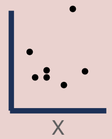
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

X = age, Y = prognostic (high=1 or low=0)

X = X / 100, Y = Y

$$f(x) = \hat{y} = w_0 x + b_0$$

$$\mathscr{L} = |\hat{y} - y|^2$$

$$x = 0.85,\ w_{0,t_0} = 0.1,\ b_{0,t_0} = 0.3\ ;\ \hat{y} = 0.38$$

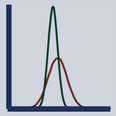
$$y = 1;\ \hat{y} = 0.38\ ;\ \mathscr{L} = 0.92$$

Compute the gradient and use to update:

$$w_{0,t_1} = 0.05\ ;\ b_{0,t_1} = 0.1$$

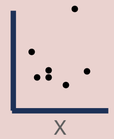
B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

3- Define your model architecture 

4- Define your loss function 

5- Predict $\hat{Y}$ from X (untrained) 

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

7- Use the loss to optimise your model 

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

10- Test on unseen data 

X = age, Y = prognostic (high=1 or low=0)

X = X / 100, Y = Y

$$f(x) = \hat{y} = w_0 x + b_0$$

$$\mathscr{L} = |\hat{y} - y|^2$$

$$x = 0.85,\ w_{0,t_0} = 0.1,\ b_{0,t_0} = 0.3\ ;\ \hat{y} = 0.38$$

$$y = 1;\ \hat{y} = 0.38\ ;\ \mathscr{L} = 0.92$$

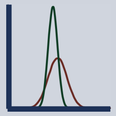
Compute the gradient and use to update:

$$w_{0,t_1} = 0.05\ ;\ b_{0,t_1} = 0.1$$

take new x, new y, compute new $\hat{y}$, …

B. **The** deep learning algorithm

1- Define your data: X and Y 

2- Preprocess your data 

X = age, Y = prognostic (high=1 or low=0)

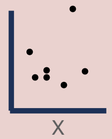
X = X / 100, Y = Y

3- Define your model architecture 

$$f(x) = \hat{y} = w_0 x + b_0$$

4- Define your loss function 

$$\mathscr{L} = |\hat{y} - y|^2$$

5- Predict $\hat{Y}$ from X (untrained) 

$$x = 0.85,\ w_{0,t_0} = 0.1,\ b_{0,t_0} = 0.3\ ;\ \hat{y} = 0.38$$

6- Compute your error (loss): $\mathscr{L} = Y - \hat{Y}$

$$y = 1;\ \hat{y} = 0.38\ ;\ \mathscr{L} = 0.92$$

7- Use the loss to optimise your model 

Compute the gradient and use to update:

$$w_{0,t_1} = 0.05\ ;\ b_{0,t_1} = 0.1$$

8- Do 5 and 6 in Validation ✔

9- Loop on 5 to 8 untill convergence wo overfitting

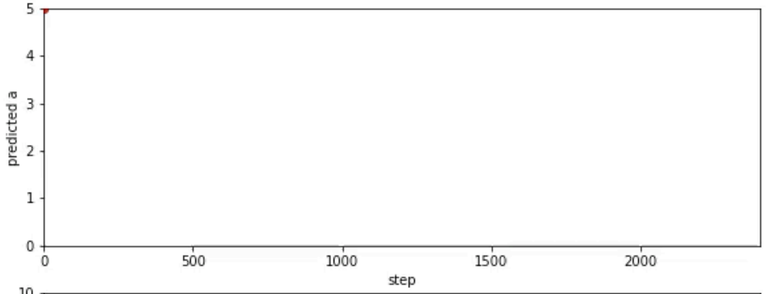
take new x, new y, compute new $\hat{y}$, …

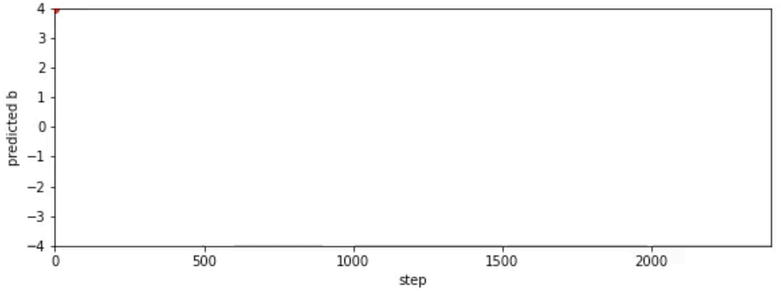
10- Test on unseen data 

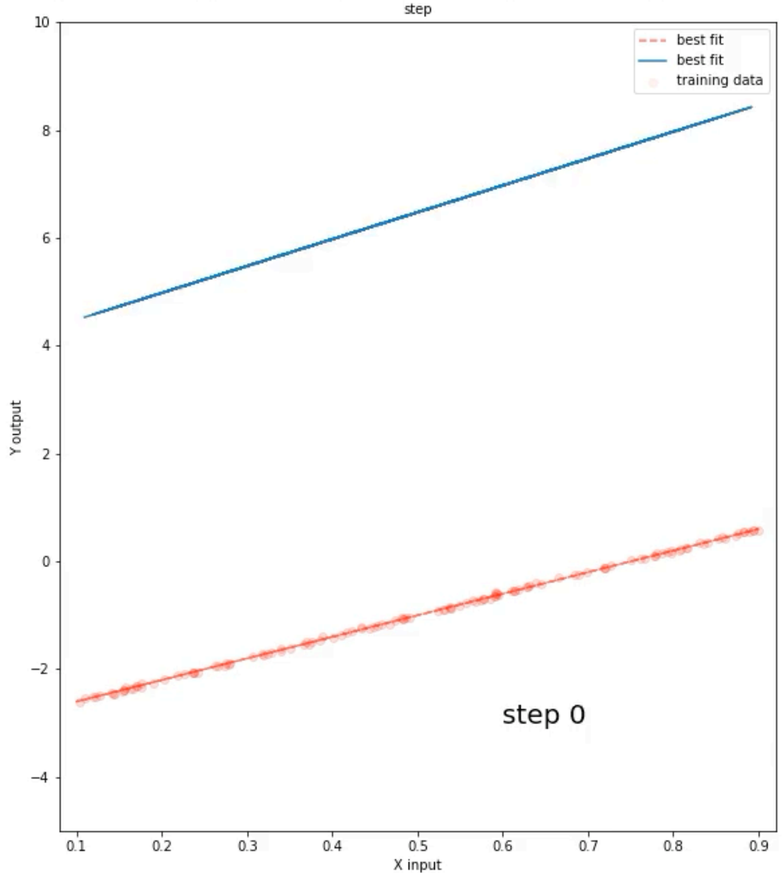
Use your trained model to predict new $\hat{y}$, check metrics (e.g. accuracy/precision here)

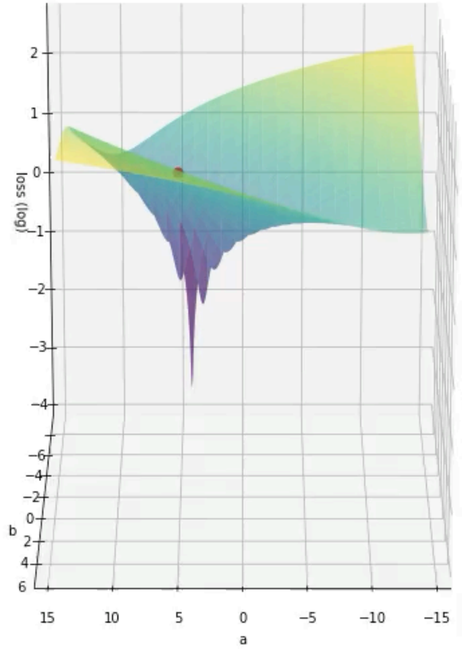
Hands-on !

1a-linear_regression.ipynb


predicted a
step


predicted b
step


best fit
best fit
training data
Y output
X input
step 0


loss (log)
b
a

Important concept

random / fix

During training, you have different sources of randomness (initialisation of the weights, order of the seen samples, selection of the training set...). But once trained, your model is deterministic: it's just a parameter function, for the same X, it will always return the same Y

Nevertheless, models can be unstable, i.e., have a training fluctuating a lot depending on this randomness. You need to test it and fix it if you want a stable training. You can also fix the seeds, such that the training is also perfectly reproducible

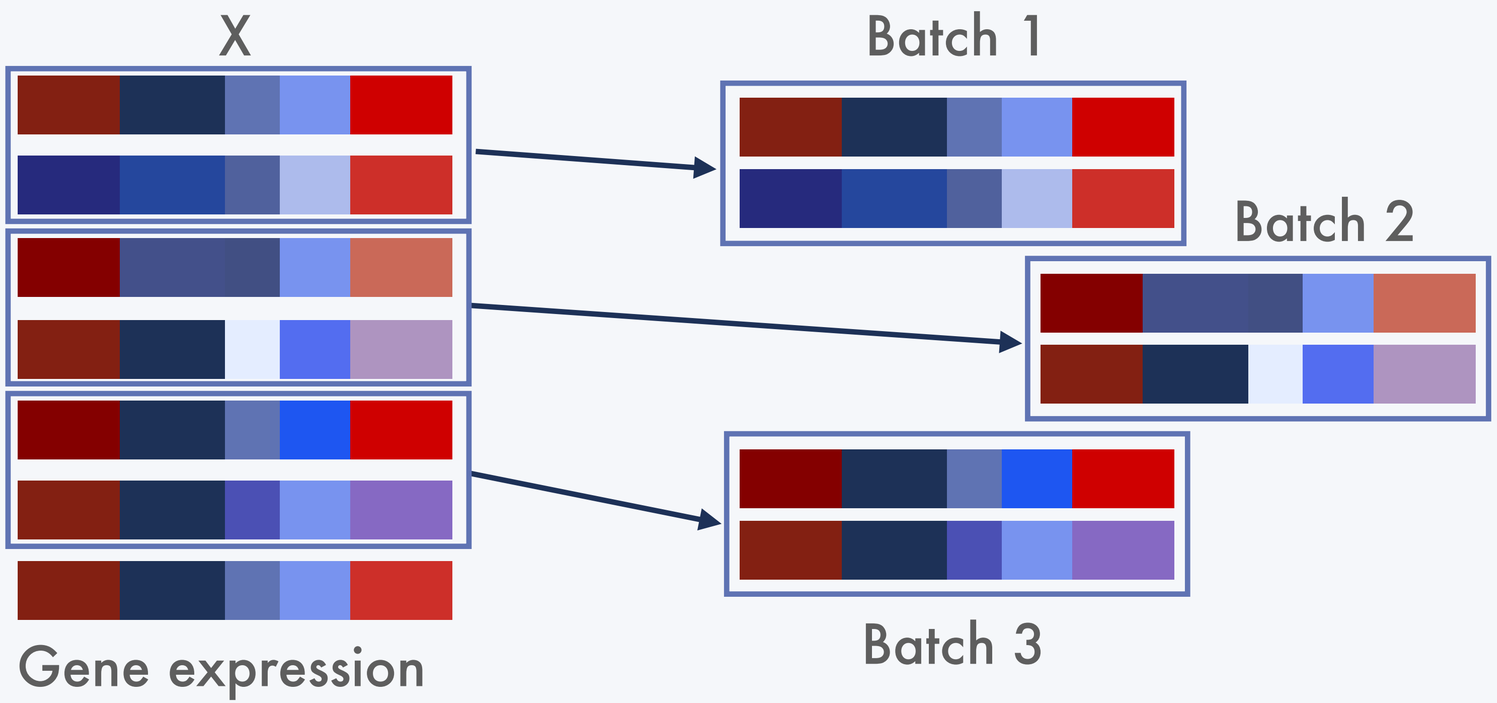
```
seed = 100

np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.enabled = False
```

Important concept

stochastic / batch

We usually don't optimise with the full training dataset at once: we do "**batch**", i.e. taking chunks of data for each optimisation step

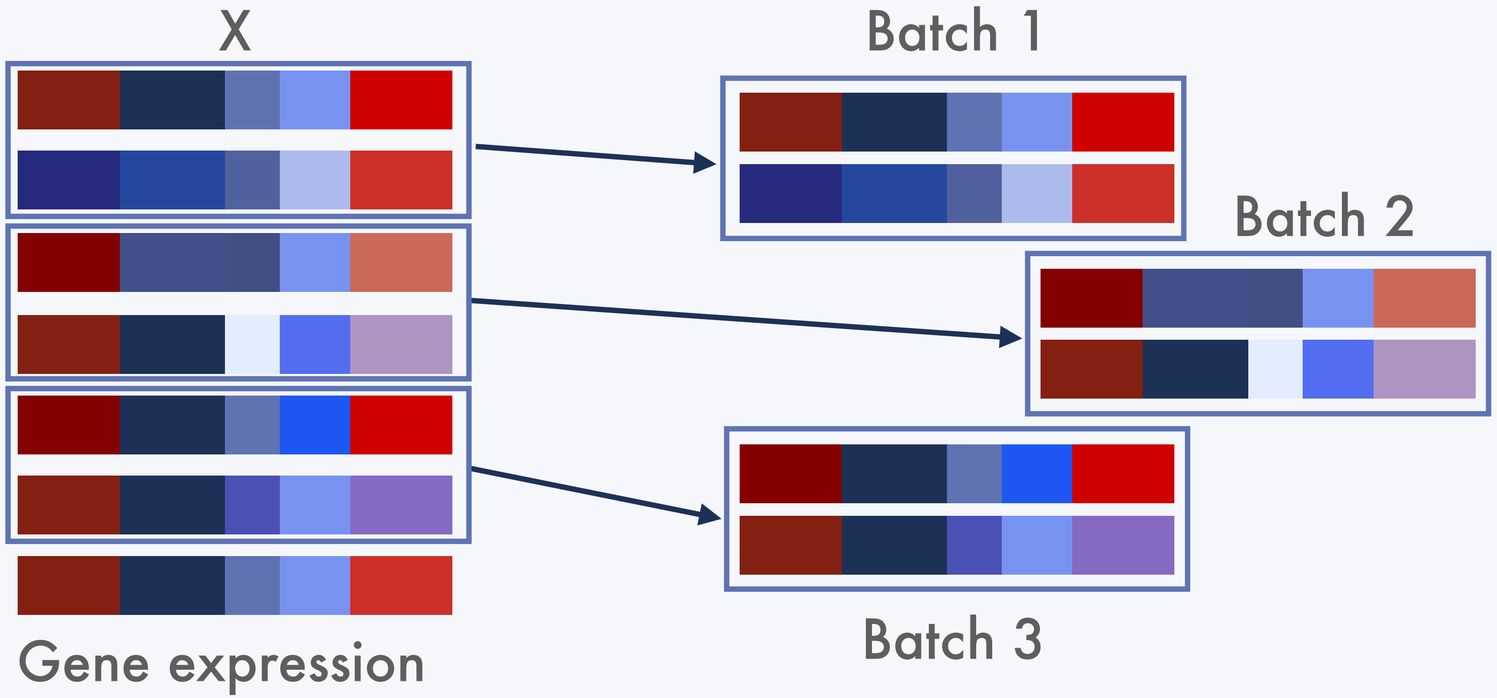


Batch can help prevent over-fitting, but also prevent memory overflow. We usually use a batch of 32, 64, or so on, depending on the size of the data

Important concept

stochastic / batch

We usually don't optimise with the full training dataset at once: we do "**batch**", i.e. taking chunks of data for each optimisation step



A step is each time we compute the gradient / optimise our weights

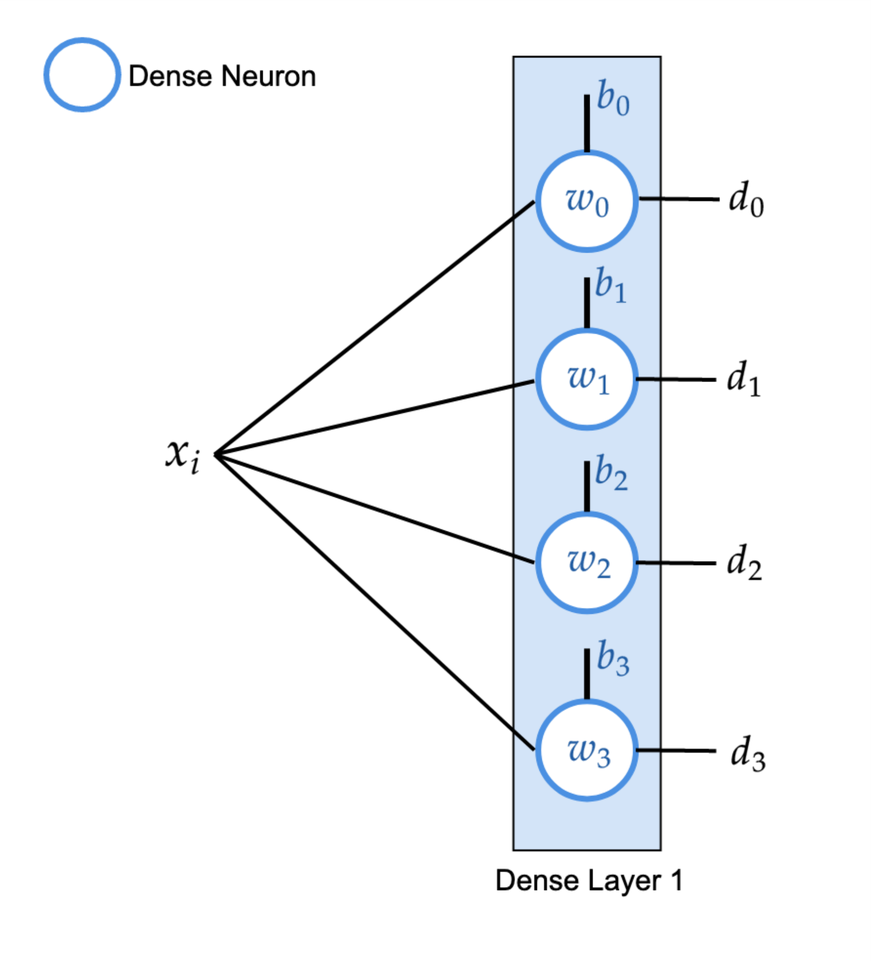
An epoch is when we have seen all our training data.
If you have 100 examples and a batch size of 10, you will have 10 steps per epoch.

Hands-on !

1b-linear_regression_stochastic.ipynb

Second example: neural network

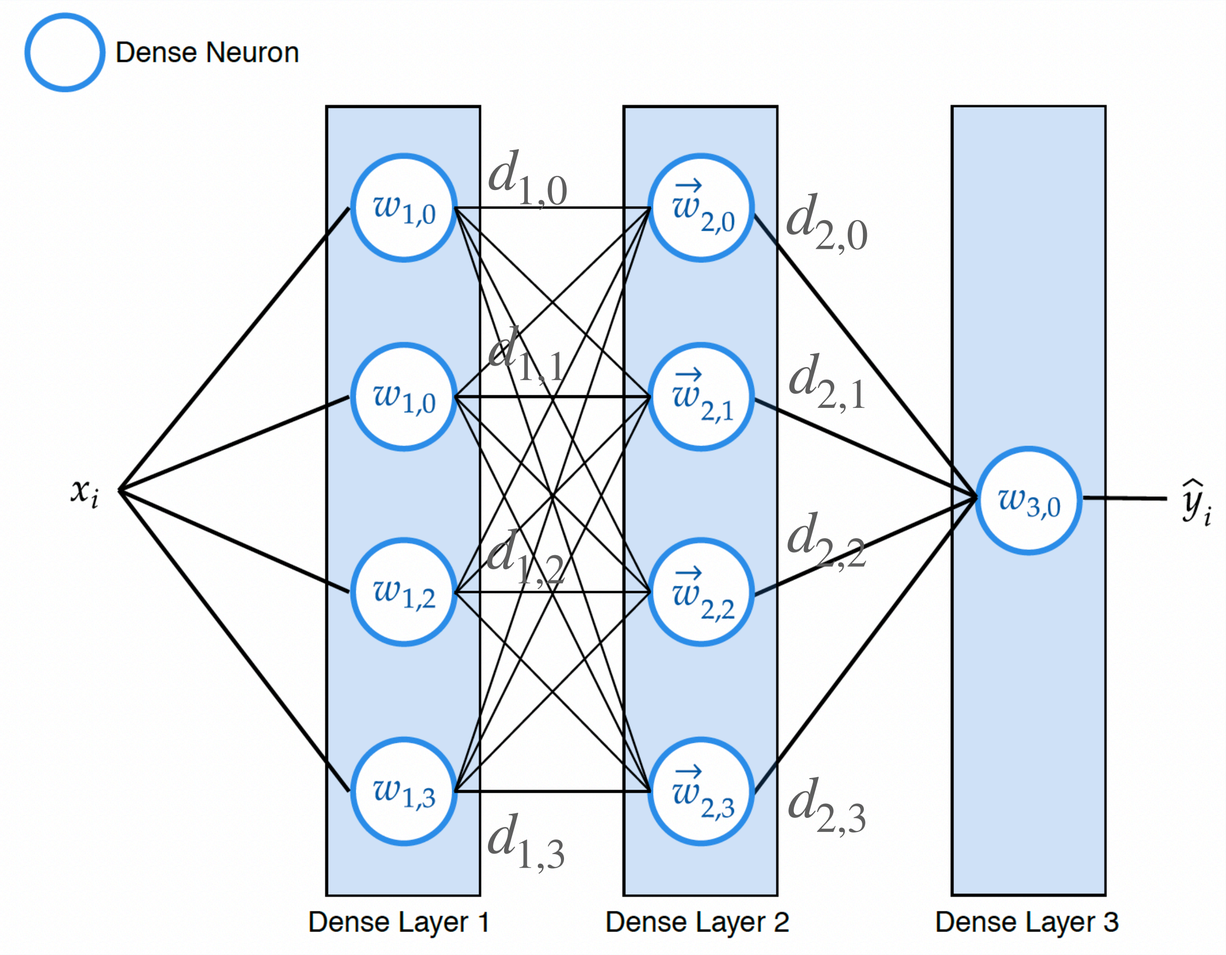
Instead of using one neuron, we can use multiple ones, stacking them "vertically". Each neuron can learn different aspects of the same data, specialising in a particular task. The association of different neurons receiving the same input is called a layer.



Second example: neural network

Instead of using one neuron, we can use multiple ones, stacking them "vertically". Each neuron can learn different aspects of the same data, specialising in a particular task. The association of different neurons receiving the same input is called a layer.

And then, we can stack neurons "horizontally", creating multiple layers. We increase a lot the complexity of the model, allowing every neuron to communicate, to learn more and more complex relations/ patterns. The intermediate representations of the data are called features or embeddings



Important concept

Activation

The perceptron that we presented in our first example is **linear**. Stacking linear layers will still be linear. This won't really build the complexity we wanted to reach. Most biological phenomena are not linear…

That's why we (almost always) add a nonlinear function to the perceptron (or neuron), called activation.

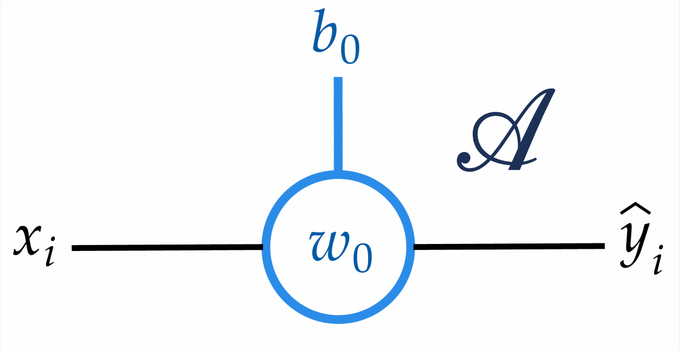
Important concept

Activation

The perceptron that we presented in our first example is **linear**. Stacking linear layers will still be linear. This won't really build the complexity we wanted to reach. Most biological phenomena are not linear…

That's why we (almost always) add a nonlinear function to the perceptron (or neuron), called activation.

$$f : x \rightarrow \mathcal{A}(w_0 x + b_0)$$



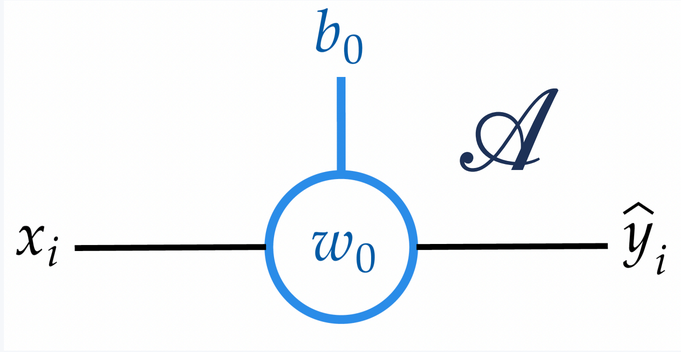
Important concept

Activation

The perceptron that we presented in our first example is **linear**. Stacking linear layers will still be linear. This won't really build the complexity we wanted to reach. Most biological phenomena are not linear…

That's why we (almost always) add a nonlinear function to the perceptron (or neuron), called activation.

$$f : x \rightarrow \mathcal{A}(w_0 x + b_0)$$



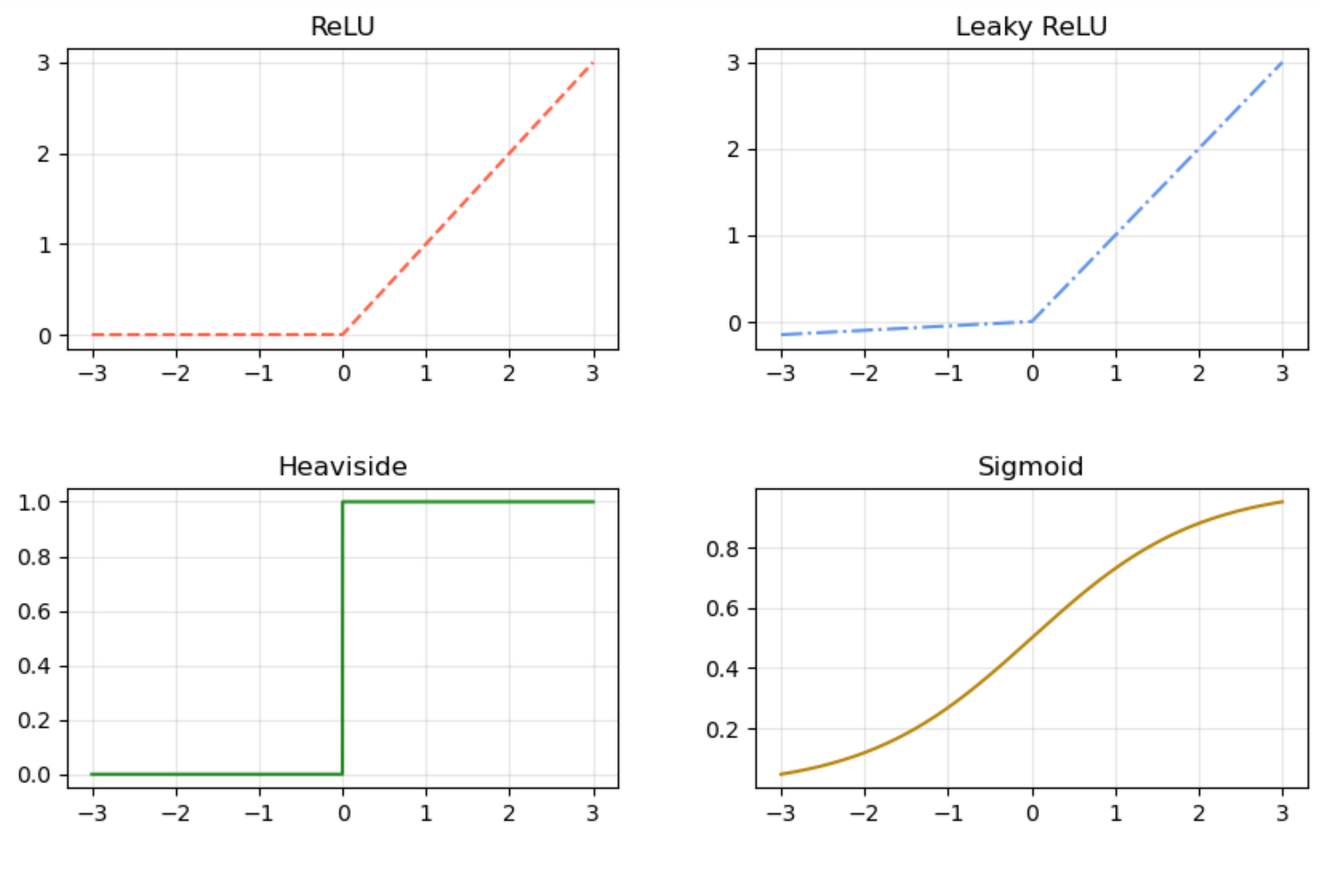
rectified linear unit (**ReLU**)

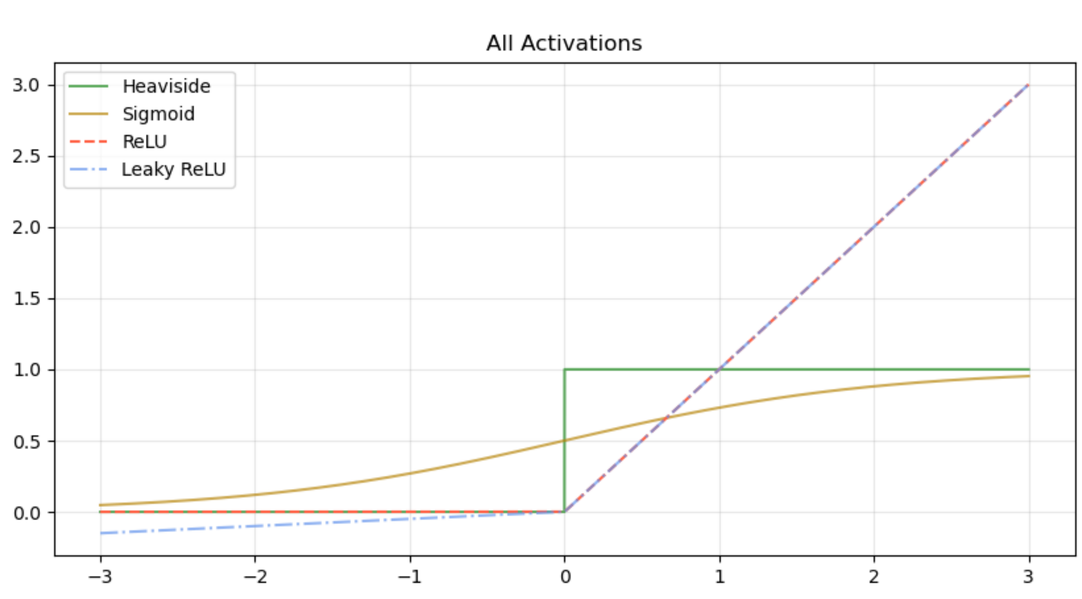
This is similar to a biological neuron, which passes the signal only if the input is "important enough", i.e. the electric current is above a certain value.
Even simple activation is most of the time enough to build complex non-linearity in your model

Important concept

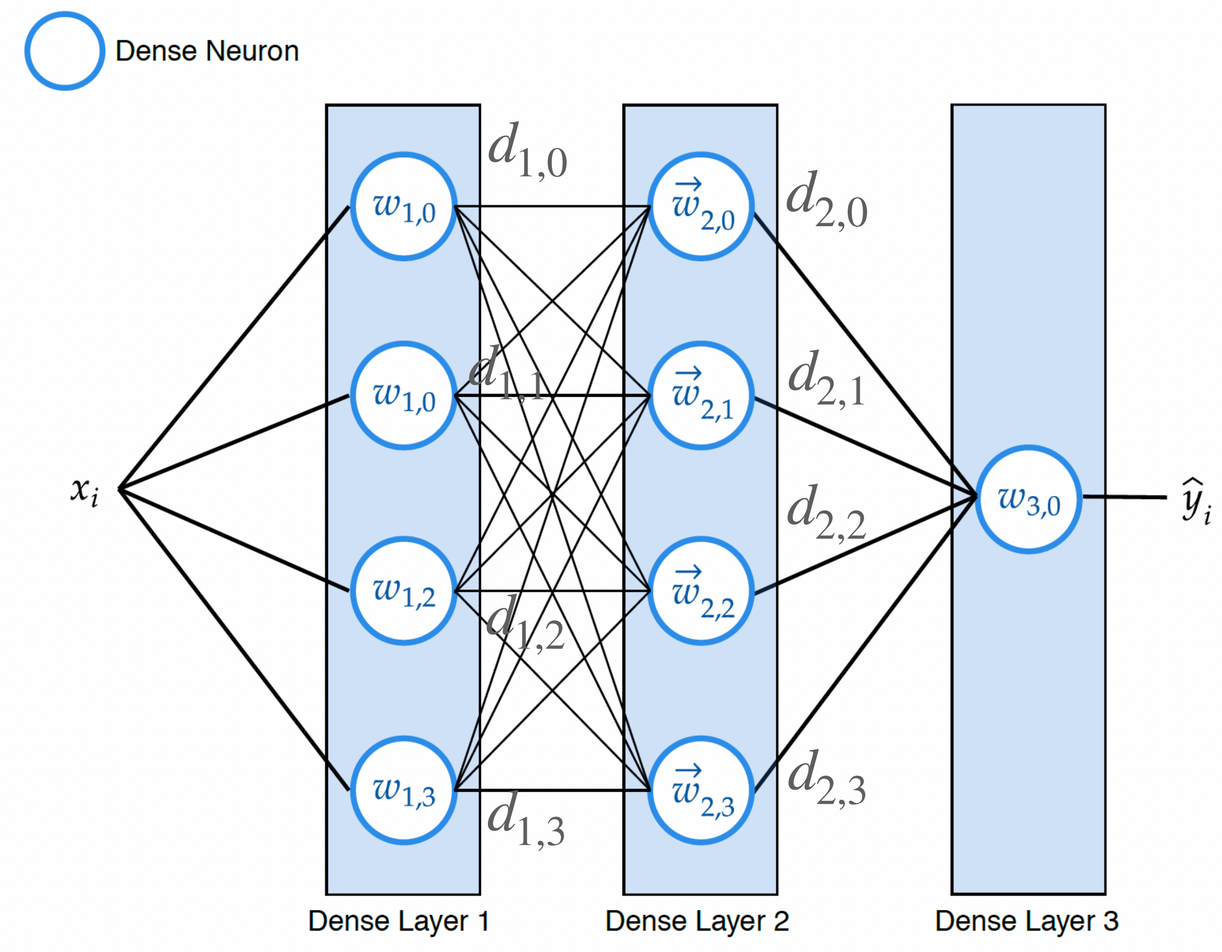
Activation

The perceptron that we presented in our first example is **linear**. Stacking linear layers will still be linear. This won't really build the complexity we wanted to reach. Most biological phenomena are not linear...





Second example: neural network



how many parameters ?

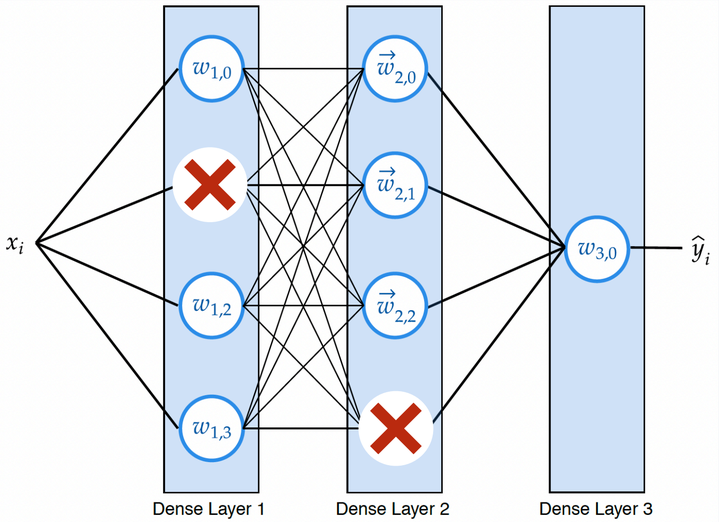
Important concept

Dropout

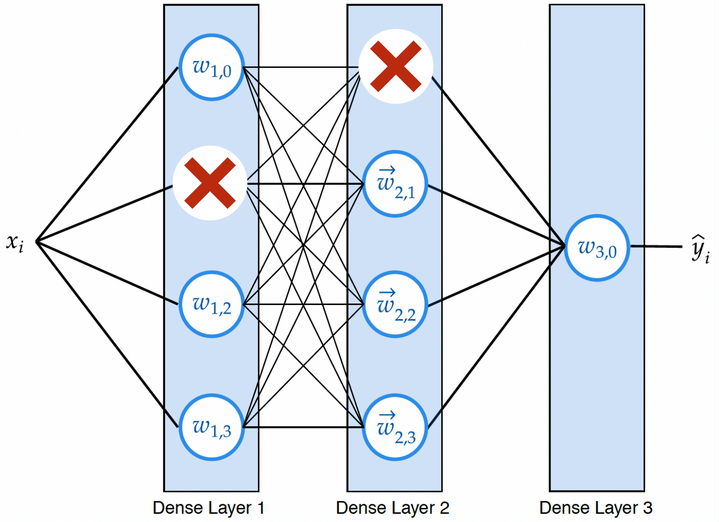
With deeper architectures, we get more complex models that can approximate very complex behaviour… but also overfit a lot. One mechanism of action against it is **dropout**.

Using dropout is setting random weights of our model to 0 during training. This way, the model learns to predict $\hat{y}$ without all the information, making it less prone to learning by heart the data. It can strongly help against overfitting. You can set the fraction of weights to set to zero, and up to 30% of dropout is known to produce great results!

Model during step i



Model during step j



Hands-on !

2-non_linear_regression.ipynb

Important concept

invertible normalisation

Normalisation is crucial for deep learning. But you need to be sure that you can, after prediction, go back to your original scale. For this, you need to be sure that your normalisation function is bijective and invertible. For example, you can’t clip values: you would not know to which unclipped value you should project new data.

Important concept

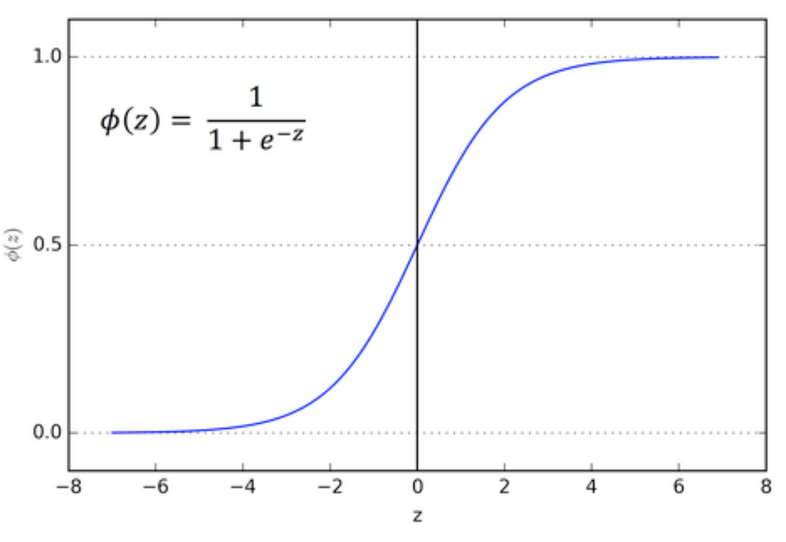
Last layer activation

As we've seen, activation is crucial to building a deep model. Without them, going deeper just means stacking linear operations, ending up with just a big linear model. Usually not enough for most of the cases. But the **last layer** is different, as it **produces the predictions** (output) of the model.

For this reason, you need to think more carefully. In general cases, you can simply omit the last activation, letting the model be able to predict the whole range of real numbers.

But you can also think smarter: for example, if you are predicting a probability (e.g. in binary classification), you may want to help your model to know that it should always predict numbers between 0 and 1. You can do it with the softmax activation.

In the other direction, if you know that your model needs to predict negative values, do not put a ReLU: the model won't be able to generate those negative values, as the ReLU sets to 0 everything that is below zero

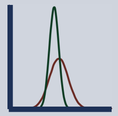


Sigmoid activation

Recap

A common pipeline

1- Define your data: X and Y

2- Preprocess your data 

3- Define your model architecture

4- Define your loss function

5- Predict $\hat{Y}$ from X (untrained)

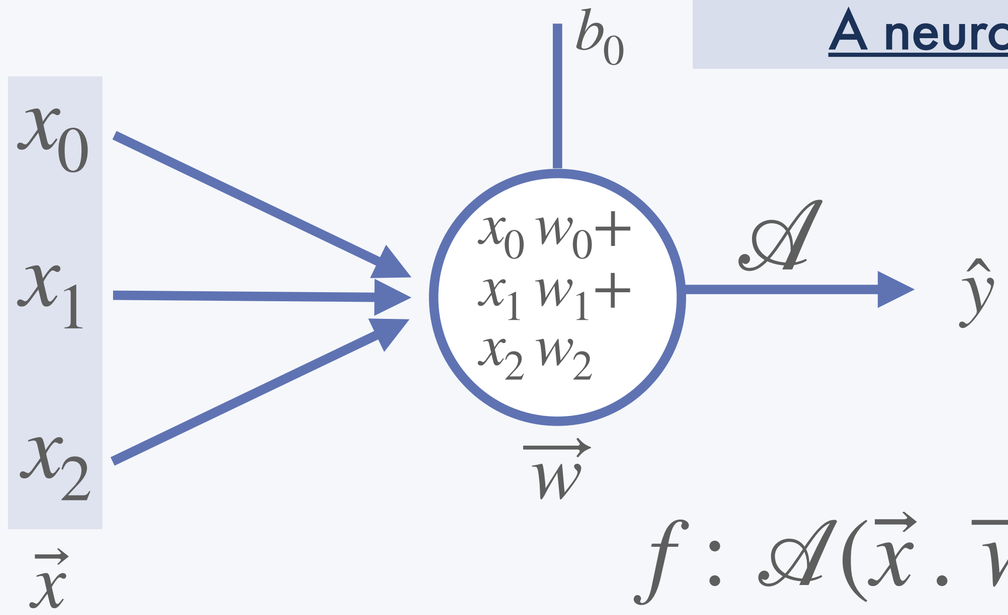
6- Compute your loss: $\mathscr{L} = |y - \hat{y}|^2$

7- Use the loss to optimise your model

8- Loop on 5 - 6 - 7 untill convergence

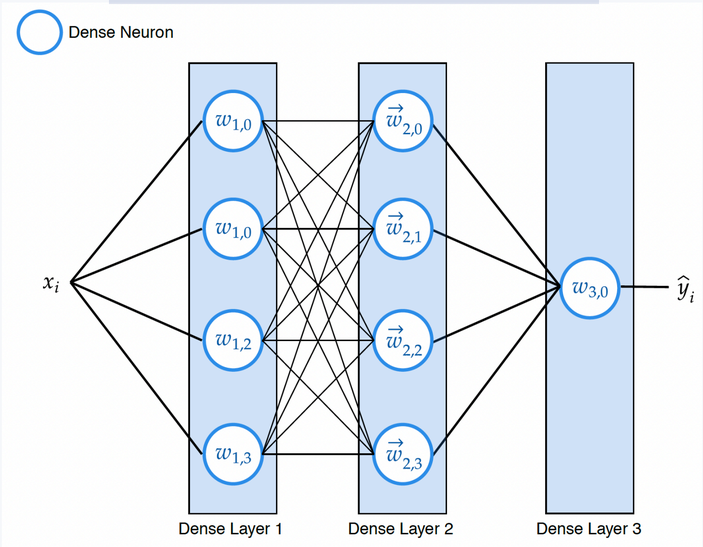
9- Test on unseen data 

A neuron



$$f : \mathscr{A}(\vec{x} \,.\, \vec{w} + b)$$

A Neural Network



Activation for non linearity